PES UNIVERSITY, BANGALORE — Department of Computer Science and Engineering

SnakeXtreme

Software Engineering Mini-Project — Deliverables Part-1 (Team 16)

Course	UE24CS341A Software Engineering — Level 3 Mini-Project (Jackfruit Phase-1)
Project	SnakeXtreme – Multi-Mode Snake Game with Power-ups & Leaderboard
Team	Team 16 — Ayushi Srivastava (PES1UG24CS105), B V Tejas (PES1UG24CS108), Bharath Hegde (PES1UG24CS114), C Dinesh Kumar (PES1UG24CS117)
Submission	Google Form deadline: 3 October 2026

Contents of this combined PDF (order: SRS + SAD + STP)

Part	Document	Deliverable
1	Software Requirements Specification v1.0 (completed earlier; unchanged) — FR-001–029, NFR-001–007, SEC-001–006, use-case diagram UCD-1	1 — SRS
1a	SRS Addendum v1.1 — Security Objectives (SO-1…4), SEC-007, use-case diagrams UCD-2/3, clarified parameters, populated traceability matrix	1 — SRS (supplement)
2	Software Architecture & Design Specification v1.0 — component diagram, layered pattern and ADRs, traceability, security architecture, 4 sequence diagrams, API design, error handling	2 — SAD
3	Software Test Plan v1.0 — sections 1–15, §5.1 security validation, §13 traceability, Appendix A with 44 test cases	3 — STP

Git repository: https://github.com/dineshkumarc-1/software_engineering_mini_project

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

for

SnakeXtreme – Multi-Mode Snake Game with Power-ups & Leaderboard

Field	Details
Version	1.0 — Initial (Requirements Stage)
Prepared by	Team 16 — Ayushi Srivastava, B V Tejas, Bharath Hegde, C Dinesh Kumar
Organization	PES University, Dept. of CSE — UE24CS341A Software Engineering
Date Created	31 August 2026

Revision History

Name	Date	Reason for Changes	Version
Team 16	31-Aug-2026	Initial SRS draft for Requirements submission	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of SnakeXtreme, a desktop, single-player Snake game developed in C++ using the SFML library. It covers functional, non-functional and security requirements for the complete system: core gameplay, game modes and power-ups, scoring and leaderboard persistence, and menu/settings navigation. This SRS is the baseline against which the Design, Implementation and Test Case deliverables of the project will be validated.

1.2 Intended Audience and Reading Suggestions

This document is intended for: the four development-team members (as designers, implementers and testers of their owned features), the course instructor/evaluator, and any peer reviewer of the project. Section 2 gives an overall description of the product; Sections 3–4 describe interfaces and analysis models; Section 5 lists detailed functional requirements grouped by feature/owner; Sections 6–7 cover non-functional, security and other requirements; the Appendices provide a glossary, file field layouts and the requirement traceability matrix.

1.3 Product Scope

SnakeXtreme digitizes and extends the classic Snake arcade concept into a replayable desktop game with three selectable game modes, three difficulty levels, three power-up types, mode-specific obstacles, a persistent local Top-10 leaderboard, and a settings system for controls/audio/theme. The product benefits the course's Agile/SE learning objectives by decomposing cleanly into four independently ownable, testable functional modules while remaining a self-contained, non-networked application with no external customer dependency.

1.4 References

- PES University — Jackfruit Mini-Project: Software Engineering Project Guidelines 2026 (UE24CS341A).
- PES University — Software Engineering Level 3 Student Submission Guide (Coffee Kiosk Exemplar).
- SFML 2.5 Official Documentation — https://www.sfml-dev.org/documentation/2.5.1/
- Team 16 Synopsis / Project Proposal, Version 1.0.

2. Overall Description

2.1 Product Perspective

SnakeXtreme is a new, self-contained desktop product; it is not a follow-on to an existing system and has no external system dependencies beyond the local operating system, filesystem and the SFML runtime libraries. The system consists of a single executable comprising a Presentation/Rendering layer (SFML window, sprites, HUD), a Game Logic layer (movement, collision, modes, power-ups, scoring), and a Persistence layer (leaderboard and settings files on local disk).

2.2 Product Functions

At a high level the product lets the player:

- Navigate a main menu to start a new game, view the leaderboard, adjust settings, or exit.
- Select a game mode (Classic / Timed Challenge / Obstacle) and a difficulty level (Easy / Medium / Hard).
- Control a snake on a grid to consume food, grow, and avoid ending the game via wall, self, or obstacle collisions.
- Collect time-limited power-ups (Speed Boost, Score Multiplier, Shrink) that temporarily modify gameplay.

- Pause and resume an active game, and view a game-over summary with the final score.
- Have qualifying scores persisted to a local Top-10 leaderboard, viewable from the main menu.
- Customize and persist control key bindings, audio volume, and visual theme.

2.3 Scope

The table below fixes the boundary of Version 1.0 so that requirements, design and testing are not written against an open-ended feature set.

In Scope	Out of Scope
Single-player, single-machine gameplay	Multiplayer / networked gameplay
Classic, Timed Challenge and Obstacle game modes	Mobile or web-based versions
Easy / Medium / Hard difficulty levels	Cloud-hosted or online leaderboard
Speed, Multiplier and Shrink power-ups	AI-controlled opponent snake
Local file-based leaderboard (Top 10) and settings persistence	In-app purchases / monetization
Keyboard-based controls, sound effects and background music	Procedurally generated levels beyond obstacle placement

2.4 User Classes and Characteristics

SnakeXtreme has a single human-facing role. The Local File System referenced elsewhere in this document (Section 3.2, Appendix B) is a persistence dependency, not a user class, and is therefore modelled only as a secondary actor in the use-case diagram, not below.

User Class	Privileges	Characteristics
Player	Start/pause/resume games; select mode and difficulty; view leaderboard; change settings	Non-technical, casual desktop user. Includes both first-time players (need an immediately learnable menu flow) and returning players (primarily interested in beating their leaderboard rank). No privilege tiers — every player has identical access to all features.

2.5 Operating Environment

The application targets Windows 10/11 (primary) desktop environments with a minimum screen resolution of 800×600, running as a native compiled C++17 executable linked against SFML 2.5+. Linux compatibility (via SFML's cross-platform support) is a stretch goal and not a Version 1.0 requirement.

2.6 Design and Implementation Constraints

- Must be implemented in C++17 using the SFML library, as assigned for this project.
- Game logic must be separated from the SFML rendering layer to remain unit-testable without a display context.
- No external network calls or backend server — all persistence is local-file-based.
- Must follow Agile methodology with sprint-wise, traceable Git/GitHub activity from every team member.
- Grid size fixed at 30×20 cells for Version 1.0 to keep collision and rendering logic consistent across modes.

2.7 Assumptions and Dependencies

- Assumes the target machine has a graphics driver compatible with SFML's OpenGL-based rendering.
- Assumes a keyboard is available; no touch or gamepad support is planned for Version 1.0.
- Depends on the SFML 2.5+ runtime and the nlohmann/json library (or equivalent) for persistence.
- Assumes single-user, single-instance execution — concurrent multi-instance leaderboard writes are out of scope.

3. External Interface Requirements

3.1 User Interfaces

The game presents a 2D SFML-rendered window (default 900×600 px) with the following screens: Main Menu, Mode/Difficulty Selection, Gameplay (with HUD showing score, mode, timer where applicable), Pause overlay, Game Over screen, Leaderboard screen, and Settings screen. All screens share a consistent navigation convention: Enter/Click to confirm, Esc to go back or pause, and arrow keys/WASD for menu and gameplay navigation. Error/validation messages (e.g., invalid name entry) are displayed as inline text below the relevant field.

3.2 Software Interfaces

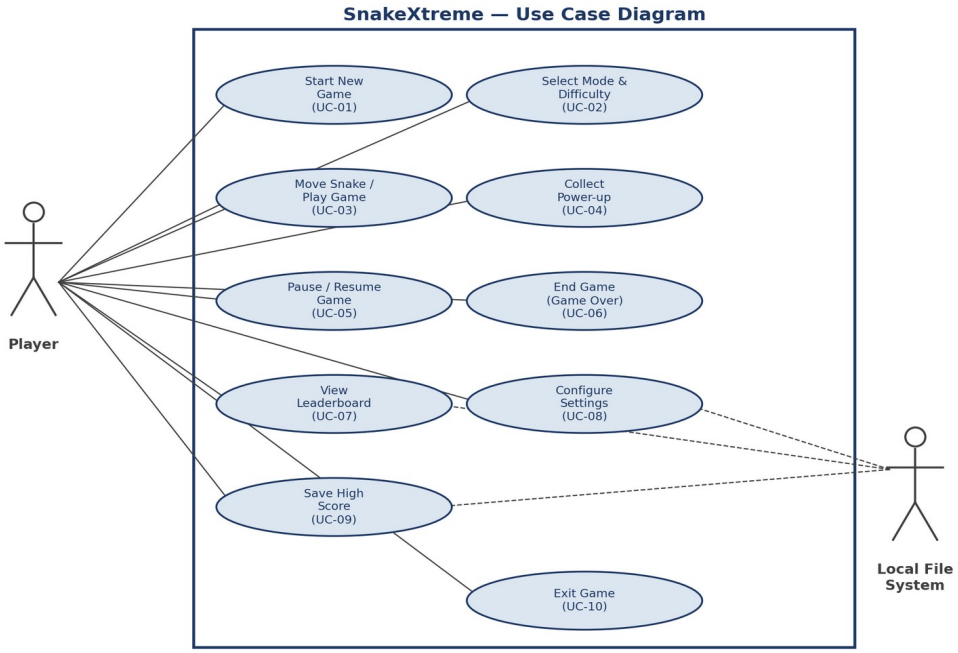
Component	Version	Purpose
SFML — Graphics module	2.5+	Window creation, 2D rendering of grid, snake, food, power-ups, UI
SFML — Audio module	2.5+	Background music and sound-effect playback
SFML — Window / System modules	2.5+	Event loop, keyboard input, timing/clock
nlohmann/json (or equivalent)	3.x	Serializing/deserializing leaderboard.json and config.json
Local Filesystem	OS-native	Persistent storage of leaderboard and settings files

3.3 Communications Interfaces

None. SnakeXtreme is a fully offline, single-machine application for Version 1.0. No network, email, or web-service communication is required or supported.

4. Analysis Models

The primary actor is the Player; the Local File System is modelled as a secondary actor representing persistence. The use case diagram below shows all major use cases identified for Version 1.0.



Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Start New Game	Player	Application launched	Player name captured; navigates to mode selection
UC-02	Select Game Mode & Difficulty	Player	UC-01 complete	Mode and difficulty set for the session
UC-03	Move Snake / Play Game	Player	Mode/difficulty selected	Snake moves, grows, and score updates in real time
UC-04	Collect Power-up	Player	Power-up present on grid	Temporary gameplay effect applied
UC-05	Pause / Resume Game	Player	Game in progress	Game loop suspended/resumed
UC-06	End Game (Game Over)	Player	Collision or timer expiry occurs	Game-over screen shown with final score
UC-07	View Leaderboard	Player	Leaderboard file exists (or defaults to empty)	Top-10 scores displayed
UC-08	Configure Settings	Player	Settings menu opened	Controls/audio/theme updated and saved
UC-09	Save High Score	Player / File System	Game over and score qualifies	Leaderboard file updated with new entry
UC-10	Exit Game	Player	Any screen active	Application terminates; settings saved

5. System Features

Functional requirements are organized below by system feature, aligned to the four owned functionalities defined in the Synopsis. Each requirement is uniquely tagged and is written to be concise, complete, unambiguous, verifiable and necessary — vague, unmeasurable phrasing (e.g. "the snake should move smoothly") is deliberately avoided in favor of concrete, testable values wherever a requirement could otherwise be interpreted subjectively.

5.1 Feature 1 — Core Gameplay Engine (Owner: Ayushi Srivastava)

Description & Priority: High. Provides the foundational grid, movement, collision and food logic on which every other feature depends.

Stimulus/Response: Player presses a direction key → snake's heading updates (unless it reverses onto itself) → on each game-loop tick the snake advances one cell → collisions and food consumption are evaluated.

ID	Requirement	Priority	Verification
FR-001	The system shall render the game grid and snake within 2 seconds of launching a new game.	High	Functional test
FR-002	The system shall advance the snake by one grid cell at a fixed tick interval per difficulty: Easy = 200 ms/cell, Medium = 120 ms/cell, Hard = 70 ms/cell.	High	Functional + timing test
FR-003	The system shall change the snake's direction based on arrow-key/WASD input, except a 180° reversal onto its own body.	High	Functional + negative test
FR-004	The system shall end the current game when the snake's head collides with a boundary wall.	High	Functional test
FR-005	The system shall end the current game when the snake's head collides with any segment of its own body.	High	Functional test
FR-006	The system shall spawn a new food item at a random unoccupied grid cell immediately after the previous food is consumed.	High	Functional test
FR-007	The system shall increase the snake's length by exactly one segment when the snake's head occupies the same cell as food.	High	Unit test

5.2 Feature 2 — Game Modes, Difficulty & Power-up/Obstacle System (Owner: B V Tejas)

Description & Priority: High. Extends the core engine with selectable play styles, scaling challenge, and reward/hazard variety.

Stimulus/Response: Player selects a mode/difficulty at UC-02 → system configures timer/obstacles/speed accordingly → during play, power-ups spawn periodically and apply timed effects when collected.

ID	Requirement	Priority	Verification
FR-008	The system shall allow the player to select one of three game modes — Classic, Timed Challenge, or Obstacle — before starting a game.	High	Functional test
FR-009	The system shall allow the player to select a difficulty level (Easy, Medium, Hard) that determines snake movement speed.	High	Functional test
FR-010	In Timed Challenge mode, the system shall end the game when the countdown timer reaches zero.	High	Functional test
FR-011	In Obstacle mode, the system shall place static obstacles on the grid that end the game upon collision with the snake's head.	High	Functional test
FR-012	The system shall spawn a power-up item at a random unoccupied	Medium	Functional test

ID	Requirement	Priority	Verification
	cell at intervals not exceeding 15 seconds during active gameplay.		
FR-013	The system shall apply a temporary speed-boost effect for 5 seconds when the snake collects a Speed power-up.	Medium	Unit + functional test
FR-014	The system shall apply a ×2 score-multiplier effect for 10 seconds when the snake collects a Multiplier power-up.	Medium	Unit test
FR-015	The system shall reduce the snake's length by one segment (minimum length 3) when the snake collects a Shrink power-up.	Medium	Unit + boundary test

5.3 Feature 3 — Scoring, Leaderboard & Persistence (Owner: Bharath Hegde)

Description & Priority: High. Owns score calculation and all local persistence, ensuring player progress survives across sessions.

Stimulus/Response: Food/power-up consumption → score recalculated and redrawn on HUD; Game over with a qualifying score → leaderboard file updated; Player opens Leaderboard → file loaded and displayed.

ID	Requirement	Priority	Verification
FR-016	The system shall increment the player's score by a mode-specific value each time food is consumed.	High	Unit test
FR-017	The system shall display the current score on-screen in real time during gameplay.	High	Functional test
FR-018	The system shall persist the Top-10 high scores (player name, score, mode, difficulty, date) to a local leaderboard file.	High	Integration test
FR-019	The system shall load and display the persisted leaderboard when the player selects 'View Leaderboard' from the main menu.	High	Functional test
FR-020	The system shall insert a new score into the leaderboard file only if it ranks within the current top 10.	High	Unit + boundary test
FR-021	The system shall validate leaderboard file structure on load and revert to a default empty leaderboard if validation fails.	Medium	Negative test

5.4 Feature 4 — Menu, UI Navigation & Player Settings (Owner: C Dinesh Kumar)

Description & Priority: High. Owns every screen transition outside active gameplay and all player-configurable preferences.

Stimulus/Response: Player launches app → main menu displayed; Player selects a menu option → corresponding screen loads; Player changes a setting → preference applied immediately and persisted.

ID	Requirement	Priority	Verification
FR-022	The system shall display a main menu with options New Game, Leaderboard, Settings, and Exit on application launch.	High	Functional test
FR-023	The system shall prompt the player to enter a display name (maximum 15 characters) before starting a new game.	High	Functional + boundary test
FR-024	The system shall allow the player to pause and resume an active game using the Esc key.	High	Functional test
FR-025	The system shall display a Game Over screen showing the final score with options to restart or return to the main menu.	High	Functional test
FR-026	The system shall allow the player to remap the four movement control keys via the Settings menu.	Medium	Functional test
FR-027	The system shall allow the player to independently adjust	Medium	Functional test

ID	Requirement	Priority	Verification
	background-music and sound-effect volume via the Settings menu.		
FR-028	The system shall allow the player to select a visual theme/skin for the snake and grid via the Settings menu.	Low	Functional test
FR-029	The system shall persist player settings (controls, volume, theme) across application restarts using a local configuration file.	High	Integration test

6. Other Non-Functional Requirements

6.1 Performance Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	The game loop shall maintain a minimum of 30 frames per second on the target hardware profile.	Performance test
NFR-002	Performance	Input-to-response latency for a direction-key change shall not exceed 100 ms.	Performance test
NFR-003	Reliability	The application shall not crash when the leaderboard or config file is missing/corrupted; it shall fall back to defaults.	Negative/ integration test
NFR-004	Usability	A new player shall be able to start their first game within 3 primary screens of launching the application.	Usability walkthrough
NFR-005	Maintainability	Game logic shall be separated into independently identifiable modules (engine, modes, persistence, UI) with unit tests.	Code review
NFR-006	Portability	The application shall build and run on Windows 10/11 using SFML 2.5+; Linux support is a stretch objective.	Build verification
NFR-007	Testability	Core gameplay logic shall be exposed via functions/classes that are testable without an active SFML rendering context.	Unit test

6.2 Safety Requirements

SnakeXtreme is a non-safety-critical desktop game; no physical or health-related safety hazards are applicable. The only safety-adjacent concern is data safety: the application must never leave the leaderboard or settings file in a corrupted, unreadable state after an unexpected termination (see SEC-004).

6.3 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Player name input shall be sanitized to reject delimiter/control characters before being written to the leaderboard file, to prevent file-format injection.	Negative/security test
SEC-002	The application shall not collect, require, or store any personally identifiable information beyond the player-entered display name.	Design review
SEC-003	Configuration and leaderboard files shall be validated against their expected schema before being parsed, to prevent malformed-file crashes.	Negative/security test
SEC-004	File write operations for leaderboard and settings data shall use a safe write pattern (write-to-temp-then-rename) to avoid data corruption on unexpected termination.	Integration/fault-injection test
SEC-005	The system shall treat all data loaded from leaderboard/config files as untrusted and shall parse it only as data, never execute or interpret it as code.	Security review
SEC-006	The system shall enforce a fixed maximum buffer size for the player-name input field to prevent buffer-overflow conditions in the underlying C++ implementation.	Boundary/security test

6.4 Software Quality Attributes

- Maintainability is prioritized over raw performance: the engine/mode/persistence/UI split should allow one module to change with minimal impact on others.
- Usability favors ease-of-learning: a first-time player should understand controls without reading external instructions.
- Reliability is prioritized for persistence: leaderboard/settings corruption must never crash the application.

6.5 Business Rules / Domain Rules

- A game session belongs to exactly one game mode and one difficulty level for its entire duration.
- Only scores from a completed game (i.e., a game-over event has occurred) are eligible for leaderboard submission.
- The snake's minimum length is 3 segments; the Shrink power-up cannot reduce it below this value.
- Obstacles are only present and only end the game in Obstacle mode.
- Only one power-up effect may be active on the snake at a time; a newly collected power-up replaces any active effect.

7. Other Requirements

- Data retention: leaderboard entries persist indefinitely on local disk until manually cleared by the player (no automatic expiry in Version 1.0).
- Localization: Version 1.0 supports English text only; no internationalization is planned.
- Reuse: SFML, and a JSON serialization library, are reused third-party components; all gameplay/persistence logic is developed by the team.
- Legal: No third-party copyrighted art/audio assets will be used without an appropriate open license; original or royalty-free assets only.

7.1 Initial Validation Specification (Planning Only)

At this Requirements stage, validation planning identifies how each requirement category will eventually be validated; execution and results are recorded later in the Implementation & Testing and System Validation deliverables.

Requirement Category	Planned Validation	Example
Functional (Core Engine)	Black-box functional tests + unit tests on movement/collision logic	Snake reverses direction only when no opposite-key input follows immediately
Functional (Modes/Power-ups)	Black-box functional tests per mode; mock-based unit tests for power-up timers	Timed mode ends exactly at 0:00; Speed boost reverts after 5s
Persistence	Integration tests with valid/corrupted/missing files	Leaderboard loads correctly; corrupted file falls back to defaults
Security	Negative/security tests on file schema and name-input sanitization	Player enters a name containing a delimiter character; input is rejected/sanitized
Performance	Frame-rate and input-latency measurement under normal load	Game loop sustains ≥30 FPS over a 5-minute play session
Usability	Task-based walkthrough with a first-time player	New player reaches first gameplay screen within 3 menu screens

Appendix A: Glossary

Term	Definition
SFML	Simple and Fast Multimedia Library — a C++ library for 2D graphics, audio, window and input handling.
Grid	The fixed 30×20 cell playing field on which the snake, food, power-ups and obstacles are positioned.
Game Mode	One of Classic, Timed Challenge, or Obstacle — determines win/lose conditions and hazards present.
Difficulty	Easy, Medium, or Hard — determines the snake's base movement speed.
Power-up	A collectible item (Speed, Multiplier, Shrink) that applies a temporary effect to the snake or score.
Obstacle	A static grid cell that ends the game on collision, present only in Obstacle mode.
Leaderboard	A locally persisted, ranked list of the Top-10 highest scores achieved across sessions.
HUD	Heads-Up Display — the on-screen overlay showing score, mode, and (where applicable) remaining time.
Config File	A local file storing the player's persisted settings (controls, volume, theme).

Appendix B: Field Layouts

Field layout for a single leaderboard entry, persisted in leaderboard.json:

Field	Length	Data Type	Description	Mandatory
player_name	15	String	Display name entered by the player before the game	Y
score	10	Integer	Final score achieved in the game	Y
game_mode	20	String (enum)	One of: Classic, Timed, Obstacle	Y
difficulty	10	String (enum)	One of: Easy, Medium, Hard	Y
date_achieved	10	Date (YYYY-MM-DD)	Date the score was achieved	Y

Field layout for player settings, persisted in config.json:

Field	Length	Data Type	Description	Mandatory
key_up / key_down / key_left / key_right	1	Key code	Remapped movement control keys	Y
music_volume	3	Integer (0–100)	Background music volume level	Y
sfx_volume	3	Integer (0–100)	Sound-effect volume level	Y
theme	10	String (enum)	One of: Classic, Neon, Retro	Y

Appendix C: Requirement Traceability Matrix (Initial)

At the Requirements stage, only the Requirement ID and description are finalized; Architecture, Design, Code and Test references are populated progressively in the Design and Implementation & Testing deliverables and are marked TBD here.

Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
FR-001–007	Core gameplay engine (movement, collision, food)	TBD	TBD	TBD	TBD	TBD
FR-008–015	Game modes, difficulty & power-ups/obstacles	TBD	TBD	TBD	TBD	TBD
FR-016–021	Scoring & leaderboard persistence	TBD	TBD	TBD	TBD	TBD
FR-022–029	Menu navigation & player settings	TBD	TBD	TBD	TBD	TBD
NFR-001–007	Performance, reliability, usability, maintainability, portability, testability	TBD	TBD	TBD	TBD	TBD
SEC-001–006	Input sanitization, data-privacy, schema validation, safe file writes, untrusted-data handling, buffer-overflow prevention	TBD	TBD	TBD	TBD	TBD

PES UNIVERSITY, BANGALORE — Department of Computer Science and Engineering

SRS Addendum

Supplement to SnakeXtreme SRS v1.0 — Version 1.1 (Security Objectives, Additional Use-Case Diagrams, Clarifications, RTM)

Project	SnakeXtreme – Multi-Mode Snake Game with Power-ups & Leaderboard
Version	1.1 (adds to v1.0 dated 31-Aug-2026)
Authors	Team 16 — Ayushi Srivastava, B V Tejas, Bharath Hegde, C Dinesh Kumar
Date	28-Sep-2026
Status	Draft for Part-1 submission (read together with SRS v1.0)

Revision History

Version	Date	Author	Change summary
1.0	31-Aug-2026	Team 16	Initial SRS (FR-001–029, NFR-001–007, SEC-001–006, UC-01–10).
1.1	28-Sep-2026	Team 16	Adds §6.3.1 Security Objectives; refines SEC-001; adds SEC-007; adds UCD-2 and UCD-3; records clarified design parameters; populates Appendix C traceability matrix.

Approvals

Role	Name	Signature / Email	Date
Course Coordinator / Product Owner			
Team Representative			

1. Purpose of this Addendum

SRS v1.0 already contains the introduction, 29 functional requirements, 7 non-functional requirements, 6 security requirements and the primary use-case diagram. To meet the Part-1 checklist and the SRS template, this addendum supplies the items that v1.0 leaves implicit or open:

- **Security Objectives** (template §5.1.1) — v1.0 lists security requirements but not the objectives they realise.
- **Additional UML use-case diagrams** — UCD-2 and UCD-3 refine UC-03…UC-09 (template asks for at least two diagrams).
- **Clarified parameters** — values that make requirements measurable and testable (e.g. score per food, Timed duration, speed-boost magnitude).
- **Requirements Traceability Matrix** — Appendix C is now populated with architecture, design and test references (Code references stay TBD until implementation).

Section numbers refer to SRS v1.0. Where this addendum and v1.0 differ, the addendum (v1.1) prevails.

2. Security (extends SRS §6.3)

6.3.1 Security Objectives

SnakeXtreme is an offline, single-user desktop game. Its attack surface is limited to (1) text typed into the name field and (2) the two JSON files read from local disk. The objectives below are scoped to that surface.

ID	Security objective	Rationale	Realised by
SO-1	Integrity of persisted data. Leaderboard and settings files always remain valid and uncorrupted, even after abnormal termination or manual editing.	Players expect scores to survive crashes; a half-written file must never destroy progress.	SEC-003, SEC-004, SEC-007
SO-2	Safe handling of untrusted input. Every value crossing a trust boundary (keyboard text, file contents) is validated and treated purely as data; it can never cause file-format injection or code execution.	Files on disk can be edited by anyone with local access; typed names are written into a file.	SEC-001, SEC-003, SEC-005, SEC-006, SEC-007
SO-3	Privacy by data minimisation. Only the player-entered display name is stored; no other personal information is collected.	Reduces privacy exposure to the minimum needed for a leaderboard.	SEC-002
SO-4	Availability and memory safety. Malformed files or over-long input never crash, hang or corrupt memory in the C++ implementation.	A game that crashes on a bad file is an availability failure; C++ needs explicit buffer discipline.	SEC-006, SEC-007, NFR-003

6.3.2 Security Requirements (updated)

SEC-001 is refined from “delimiter/control characters” to an explicit whitelist so that it is testable. SEC-007 is new. All others are unchanged from v1.0 with acceptance criteria added.

ID	Requirement (shall …)	Objective	Priority	Acceptance criteria / test
SEC-001	Player-name input shall be restricted to the characters A–Z, a–z, 0–9, space, underscore and hyphen; any other character (including delimiter and control characters) shall be rejected before it reaches the leaderboard file.	SO-2	High	100 % of disallowed characters rejected with inline message; file unchanged. TC-SEC-01
SEC-002	The application shall not collect, require or store any personally identifiable information beyond the player-entered display name.	SO-3	Medium	Files/log contain only Appendix B fields; no username, hostname or absolute path. TC-SEC-02
SEC-003	Configuration and leaderboard files shall be validated against their expected schema (structure, types, ranges, enums) before use.	SO-1, SO-2	High	All malformed fixtures fall back to defaults; no crash; load ≤ 1 s. TC-SEC-03
SEC-004	File write operations shall use write-to-temp-then-rename.	SO-1	High	After fault injection at 4 points the file is always old-valid or new-valid. TC-SEC-04
SEC-005	Data loaded from files shall be treated as untrusted and parsed only as data, never executed or interpreted as code or format strings.	SO-2	High	0 banned-API hits in CI scan; hostile strings displayed literally. TC-SEC-05
SEC-006	The player-name input field shall enforce a fixed maximum of 15 characters (fixed-capacity buffer).	SO-2, SO-4	High	10 000-character injection stores ≤ 15; no sanitiser errors. TC-SEC-06
SEC-007 (new)	The system shall refuse to parse a leaderboard or configuration file larger than 64 KiB and shall fall back to defaults.	SO-1, SO-4	Medium	64 KiB + 1 byte fixture rejected without full read. TC-SEC-03

3. Additional UML Use-Case Diagrams (extends SRS §4)

The original diagram (SRS §4, UCD-1) shows the ten top-level use cases. UCD-2 and UCD-3 decompose them using «include» and «extend» relationships; they introduce no new user-visible functionality, only structure that is later mapped to design components.

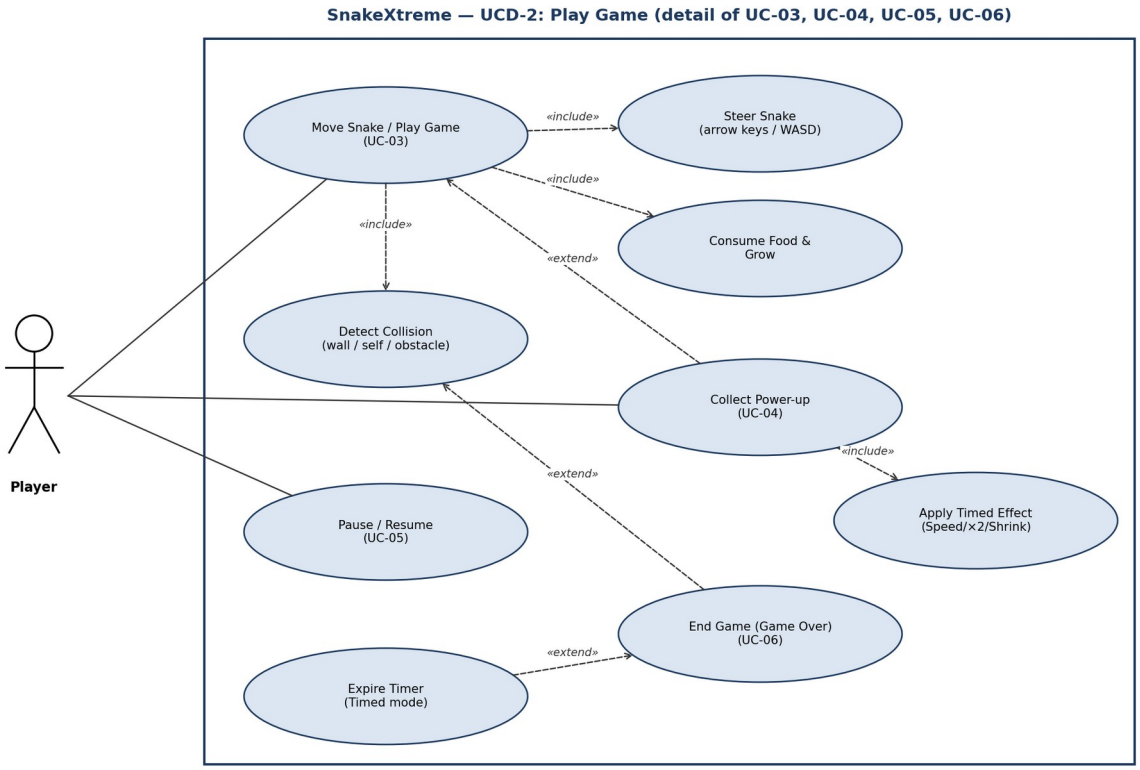


Figure 1 — UCD-2: Play Game (refines UC-03, UC-04, UC-05, UC-06)

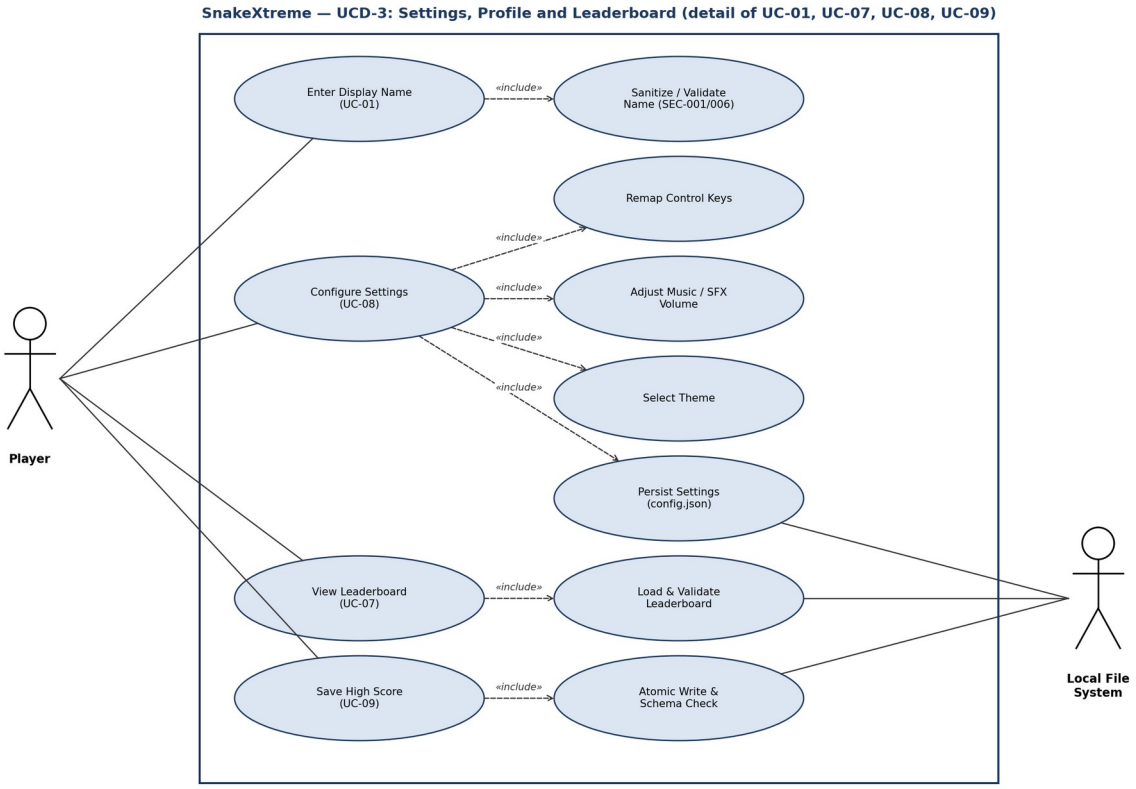


Figure 2 — UCD-3: Settings, Profile and Leaderboard (refines UC-01, UC-07, UC-08, UC-09)

Diagram	Relationship	Meaning
UCD-2	Move Snake «include» Steer Snake / Consume Food & Grow / Detect Collision	These sub-behaviours occur on every play session (FR-003–007).
UCD-2	Collect Power-up «extend» Move Snake	Optional: only when the head reaches a power-up (FR-012–015).
UCD-2	End Game «extend» Detect Collision; Expire Timer «extend» End Game	Game over is triggered by a fatal collision or, in Timed mode, timer expiry (FR-004/005/010/011).
UCD-3	Enter Display Name «include» Sanitize / Validate Name	Every name entry is validated (FR-023, SEC-001, SEC-006).
UCD-3	Configure Settings «include» Remap Keys / Adjust Volume / Select Theme / Persist Settings	FR-026–029.
UCD-3	View Leaderboard «include» Load & Validate; Save High Score «include» Atomic Write & Schema Check	FR-018–021, SEC-003, SEC-004. Local File System is the secondary actor.

4. Clarified Parameters (refines SRS §5)

The requirements below were written with intentionally measurable wording, but a few parameters were left unstated. The values in this table are the baseline used by the Design (SAD §4.1) and Test Plan (STP Appendix A). They are proposals for the product owner to confirm; any change is a one-line edit in `GameConfig`.

Parameter	Baseline value	Requirement(s)
Grid	30 × 20 cells; cell = 30 px; window 900 × 600 px	SRS 2.6, 3.1
Initial snake	3 segments, centre of grid, heading right	FR-015, TC-CORE-01
Tick interval	Easy 200 ms · Medium 120 ms · Hard 70 ms	FR-002

Parameter	Baseline value	Requirement(s)
Timed Challenge duration	60 s	FR-010
Food points	Classic 10 · Timed 10 · Obstacle 15 (before multiplier)	FR-016
Speed boost	tick interval × 0.6 for 5 s	FR-013
Multiplier	× 2 on food points for 10 s	FR-014
Shrink	−1 segment, floor at 3	FR-015
Power-up spawn gap	random 8–15 s after previous spawn; one item on grid; uncollected item is relocated	FR-012
Obstacle count	8 static cells, ≥ 3 cells from snake head, never on food	FR-011
Name rule	1–15 chars from [A-Za-z0-9 _-], trimmed	FR-023, SEC-001/006
Leaderboard tie rule	Equal score ranks below existing entry; must strictly beat 10th when full	FR-020
Default settings	Arrow keys (WASD alias always on) · music 70 · SFX 70 · theme Classic	FR-026–029
File size cap	leaderboard.json / config.json ≤ 64 KiB	SEC-003, SEC-007
Direction queue	≤ 2 pending inputs, validated against last queued heading	FR-003

Interpretation notes

- **NFR-002 (input latency ≤ 100 ms):** “response” is measured as the time until the engine has registered the direction change. The visible turn occurs at the next tick, which is bounded by the difficulty tick interval (up to 200 ms on Easy). Confirmation requested (SAD OI-1).
- **NFR-001 (“target hardware profile”):** defined in STP §6 as a Windows 10/11 x64 PC, dual-core ≥ 2.0 GHz, 4 GB RAM, integrated GPU with OpenGL 3.3.
- **FR-005 (self collision):** moving into the cell the tail is vacating on the same tick is not a collision unless the snake is growing (SAD OI-2).
- **Persistence file name:** `leaderboard.json` (SRS Appendix B) supersedes `leaderboard.dat` in the Synopsis §4.1.

5. Requirements Traceability Matrix (replaces SRS Appendix C)

Architecture references are component IDs from SAD §3.4; design references are SAD sections/sequence diagrams; test cases are defined in STP Appendix A. “Code File Ref” and “System Test ID” are completed during Implementation & Testing.

Req ID	Brief description	Architecture ref	Design ref	Test case ID(s)	Code ref
FR-001	Render grid + snake ≤ 2 s	C1, C2, C4, C6	§4.1, §4.5	TC-CORE-01	TBD
FR-002	Tick interval per difficulty (200/120/70 ms)	C6, C7, C15	§4.1, SD-1	TC-CORE-02	TBD
FR-003	Direction control; no 180° reversal	C3, C6	§4.1, SD-1	TC-CORE-03, TC-CORE-04	TBD
FR-004	Wall collision ends game	C6, C7	§4.1, SD-1	TC-CORE-05	TBD
FR-005	Self collision ends game	C6	§4.1, SD-1	TC-CORE-06	TBD
FR-006	Food spawns on random free cell	C6, C15	§4.1	TC-CORE-07	TBD
FR-007	Growth by exactly one segment	C6	§4.1, SD-1	TC-CORE-08	TBD
FR-008	Select mode (Classic/Timed/Obstacle)	C2, C7	§4.5	TC-MODE-01	TBD
FR-009	Select difficulty (Easy/Medium/Hard)	C2, C7	§4.5	TC-MODE-02	TBD

Req ID	Brief description	Architecture ref	Design ref	Test case ID(s)	Code ref
FR-010	Timed mode ends at 0	C6, C7	§4.1, SD-1	TC-MODE-03	TBD
FR-011	Obstacles fatal in Obstacle mode	C6, C7	§4.1	TC-MODE-04	TBD
FR-012	Power-up spawn interval ≤ 15 s	C8, C15	SD-3	TC-PWR-01	TBD
FR-013	Speed boost 5 s	C6, C8	SD-3	TC-PWR-02, TC-PWR-05	TBD
FR-014	×2 multiplier 10 s	C8, C9	SD-3	TC-PWR-03, TC-PWR-05	TBD
FR-015	Shrink by one (min length 3)	C6, C8	SD-3	TC-PWR-04	TBD
FR-016	Mode-specific score per food	C7, C9	§4.1, SD-1	TC-SCORE-01	TBD
FR-017	Real-time score on HUD	C2, C4, C9	§4.5	TC-SCORE-02	TBD
FR-018	Persist Top-10 with all fields	C11, C13	§4.3, SD-2	TC-LB-01	TBD
FR-019	View leaderboard from main menu	C2, C11	§4.3, §4.5	TC-LB-02	TBD
FR-020	Insert only if within Top-10	C11	SD-2	TC-LB-03	TBD
FR-021	Validate on load; default if invalid	C11, C13, C14	SD-2, §4.4	TC-LB-04	TBD
FR-022	Main menu options	C1, C2	§4.5	TC-UI-01	TBD
FR-023	Name entry, max 15 chars	C2, C10	§4.5, SD-2	TC-UI-02	TBD
FR-024	Pause/resume with Esc	C1, C2, C3	§4.5	TC-UI-03	TBD
FR-025	Game Over screen with options	C2	§4.5, SD-2	TC-UI-04	TBD
FR-026	Remap four movement keys	C2, C3, C12	§4.3, §4.5	TC-SET-01	TBD
FR-027	Independent music/SFX volume	C2, C5, C12	§4.5, SD-4	TC-SET-02	TBD
FR-028	Select visual theme	C2, C4, C12	§4.5	TC-SET-03	TBD
FR-029	Persist settings across restarts	C12, C13	SD-4	TC-SET-04	TBD
NFR-001	≥ 30 FPS	C1, C4, C6	§3.5 (ADR-002)	TC-PERF-01	TBD
NFR-002	Input latency ≤ 100 ms	C1, C3, C6	§3.5, SD-1	TC-PERF-02	TBD
NFR-003	No crash on missing/corrupt files	C11, C12, C13, C14	§4.4, SD-2, SD-4	TC-REL-01, TC-LB-04	TBD
NFR-004	First game within 3 screens	C1, C2	§4.5	TC-USE-01	TBD
NFR-005	Modular structure with unit tests	All layers	§3.5 (ADR-001)	TC-MNT-01	TBD
NFR-006	Builds/runs on Windows 10/11	Build (CMake)	§3.6	TC-PORT-01	TBD
NFR-007	Logic testable without SFML context	C6–C10, C15	§3.5 (ADR-001)	TC-TEST-01	TBD
SEC-001	Sanitise player name	C2, C10	§3.9, SD-2	TC-SEC-01	TBD
SEC-002	No PII beyond display name	C11, C12, C14	§3.9, §4.4	TC-SEC-02	TBD
SEC-003	Schema-validate files before use	C11, C12, C13	§3.9, SD-4	TC-SEC-03	TBD
SEC-004	Write-temp-then-rename	C13	§3.9, SD-2	TC-SEC-04	TBD
SEC-005	File data is data, never code	C11, C12, C13	§3.9	TC-SEC-05	TBD
SEC-006	Fixed max buffer for name input	C2, C10	§3.9, §4.3	TC-SEC-06	TBD
SEC-007	Refuse files > 64 KiB (new, Addendum)	C13	§3.9	TC-SEC-03	TBD

PES UNIVERSITY, BANGALORE — Department of Computer Science and Engineering

Software Architecture & Design Specification

SnakeXtreme – Multi-Mode Snake Game with Power-ups & Leaderboard

Project	SnakeXtreme (Team 16)
Version	1.0
Authors	Team 16 — Ayushi Srivastava, B V Tejas, Bharath Hegde, C Dinesh Kumar
Date	28-09-2026
Status	Draft — Part-1 submission
Format	IEEE 42010 / IEEE 1016 style, following the course SAD template

Revision History

Version	Date	Author	Change summary
1.0	28-Sep-2026	Team 16	Initial architecture and design (components, ADRs, security architecture, 4 sequence diagrams, internal API, error handling).

Approvals

Role	Name	Signature / Date
Course Coordinator / Product Owner		
Team Representative		

1. Introduction

1.1 Purpose — This document specifies the architecture and detailed design of SnakeXtreme v1.0, a single-player desktop Snake game in C++17/SFML. It explains how the SRS requirements are realised by components, patterns, interfaces and data formats, and how security and testability are designed in.

1.2 Scope — Covers the single executable: presentation, game logic and local persistence (leaderboard and settings). No network, server or multi-user aspects exist (SRS 3.3).

1.3 Audience — Developers (four feature owners), QA engineers, security reviewers, the course instructor/product owner, and future maintainers.

1.4 Definitions — See Appendix 5.1. Key terms: *tick* (one snake movement step), *render frame* (one window redraw), *ADR* (architecture decision record), *Repository* (class that hides file persistence), *Result<T>* (value-or-error return type), *STRIDE* (threat-classification model).

2. Document Overview

2.1 How to use this document — Section 3 describes the architecture (goals, component diagram, pattern, technology, risks, requirement traceability, security architecture). Section 4 describes the design (class-level overview, sequence diagrams, internal API and file formats, error handling, UX). Component IDs (C1–C15), sequence-diagram IDs (SD-1…SD-4) and ADR IDs are used as cross-references from the SRS Addendum RTM and the Test Plan.

2.2 Related documents — SRS v1.0 and SRS Addendum v1.1 (requirements and clarified parameters); Software Test Plan v1.0 (test cases TC-*); Team 16 Synopsis v1.0 (ownership and sprint plan).

3. Architecture

3.1 Goals & Constraints

Goals

- **Responsive and smooth:** ≥ 30 FPS and input acknowledged ≤ 100 ms (NFR-001/002).
- **Modular and maintainable:** engine, modes, persistence and UI can change independently; each has one owner (NFR-005).
- **Testable:** all game rules run without a window (NFR-007).
- **Robust persistence:** bad or interrupted file I/O never crashes the game or corrupts data (NFR-003, SEC-003/004).
- **Secure by design for its small attack surface:** validated input, untrusted-file handling, minimal data (SEC-001–007).

Constraints

- C++17 with SFML 2.5+; single Windows 10/11 executable built with CMake (SRS 2.5, 2.6).
- Game logic separated from rendering; no network; local JSON files only.
- Fixed 30×20 grid, 900×600 window, English text only.
- Agile delivery by four owners in six sprints; every module must be independently demonstrable.

3.2 Stakeholders & Concerns

Stakeholder	Concerns addressed by the architecture
Player	Responsive controls, learnable menus, scores that are never lost or corrupted.
Course instructor / product owner	Traceability requirement → component → test; evidence of clean modular design and per-member ownership.
Development team (4 owners)	Clear module boundaries and interfaces so features can be built and tested in parallel.
QA / testers	Headless testable logic, injectable Clock and RNG, deterministic fixtures.
Security reviewer	Explicit trust boundaries, input validation, safe file handling, no unsafe APIs.

3.3 Component (UML) Diagram

The system is a single process with three layers plus cross-cutting services. Arrows show compile-time dependencies (they point downwards: presentation → logic → persistence). Logic and persistence layers contain no SFML types.

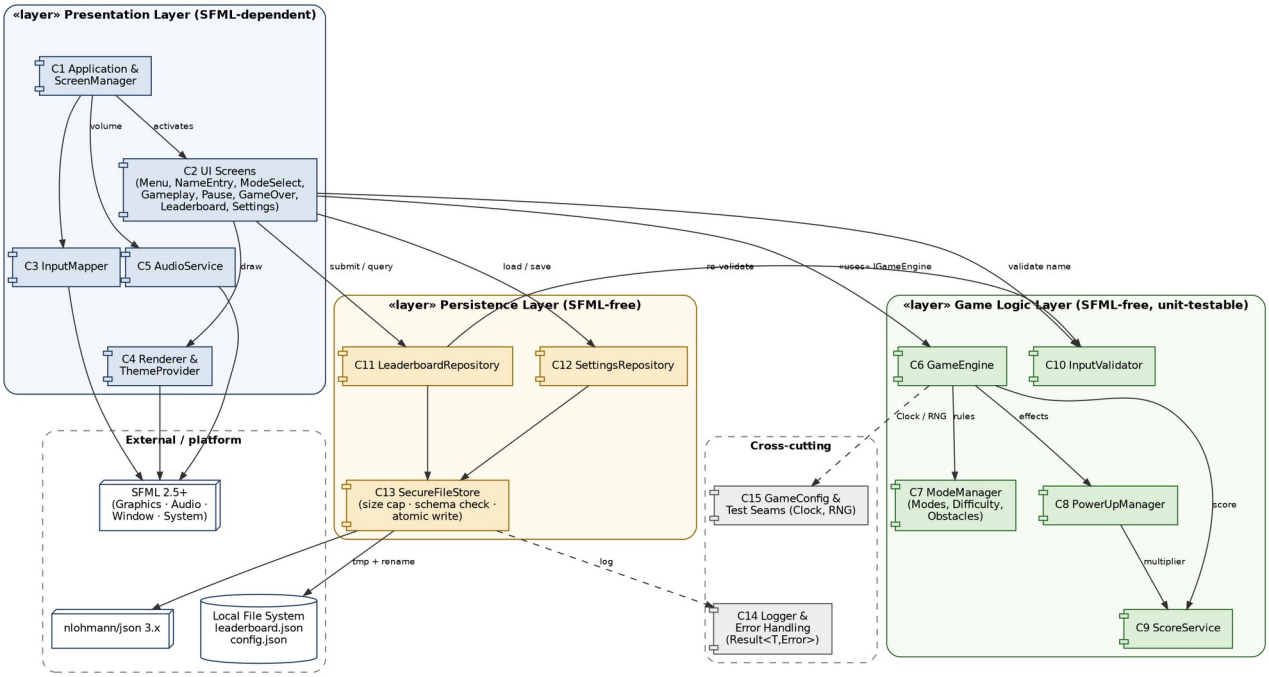


Figure 1 — SnakeXtreme component diagram (UML component view, layered)

3.4 Component Descriptions

ID	Component	Responsibilities	Key interface	Depends on	Owner
C1	Application & ScreenManager	Window, main loop (fixed-timestep), screen stack/state machine, top-level error handling, start-up load and clean shutdown (UC-01, UC-10).	`run()`, `IScreen` push/pop	C2, C3, C5, C12, C14	Dinesh
C2	UI Screens	Main menu, name entry, mode/difficulty, gameplay screen, pause overlay, game over, leaderboard, settings; inline validation messages.	`IScreen::handle/update/draw`	C4, C6, C10, C11, C12	Dinesh
C3	InputMapper	Translates SFML events and key bindings (arrows/WASD alias/remapped keys) into logical `Action`s.	`map(sf::Event)→Action`	SFML, C12	Dinesh
C4	Renderer & ThemeProvider	Draws grid, snake, food, power-ups, obstacles, HUD; loads Classic/Neon/Retro themes.	`draw(GameState, Theme)`	SFML	Dinesh
C5	AudioService	Music and SFX playback; independent volumes.	`playSfx`, `setVolumes`	SFML Audio	Dinesh
C6	GameEngine	Grid, snake, direction queue, tick loop, collisions, food spawn and growth, session lifecycle and events.	`IGameEngine` (§4.3.1)	C7, C8, C9, C15	Ayushi
C7	ModeManager	Mode strategies (Classic/Timed/Obstacle), difficulty profiles, obstacle placement, time limit, food points.	`IModeStrategy`	C15	Tejas
C8	PowerUpManager	Spawn schedule (≤ 15 s), pick-up detection, single active timed effect (Speed/Multiplier/Shrink).	`IPowerUpManager`	C6 (context), C9, C15	Tejas
C9	ScoreService	Score accumulation, multiplier, per-mode points.	`onFoodEaten`, `setMultiplier`	—	Bharath

ID	Component	Responsibilities	Key interface	Depends on	Owner
C10	InputValidator	Name whitelist and length rule (SEC-001/006); reused by C2 and C11.	`validateName`	—	Bharath
C11	LeaderboardRepository	Load/validate, rank, insert-if-top-10, persist leaderboard.	`ILeaderboardRepository`	C10, C13	Bharath
C12	SettingsRepository	Load/validate/save keys, volumes, theme; defaults.	`ISettingsRepository`	C13	Dinesh
C13	SecureFileStore	Size-capped read, JSON parse, schema validation, atomic write (temp + rename), stale-temp cleanup.	`ISecureFileStore`	nlohmann/json, file system, C14	Bharath
C14	Logger & Error Handling	`Result<T,Error>`, error codes, leveled log file without sensitive data.	`log()`, `Error`	—	Shared
C15	GameConfig & Test Seams	Constants (SRS Addendum §4), `IClock`, `IRandom` for deterministic tests.	`IClock`, `IRandom`	—	Ayushi

3.5 Chosen Architecture Pattern and Rationale

Pattern: Layered architecture (Presentation → Game Logic → Persistence) with strict downward dependencies and a SFML-free core. It gives the clean four-way ownership split (Engine, Modes, Persistence, UI) required by the Synopsis and directly satisfies NFR-005 and NFR-007. Supporting patterns keep each layer simple:

- **Game Loop with fixed timestep** — rendering runs every frame; movement advances only when the accumulated time reaches the tick interval (ADR-002).
- **State** — screens form a state machine (SAD §4.5).
- **Strategy** — game modes and power-up effects are interchangeable strategies, so adding a mode/effect does not change the engine (ADR-003).
- **Repository** — file persistence hidden behind interfaces; tests substitute a temp directory (ADR-004).

ADR	Decision	Alternatives considered	Rationale / consequences
ADR-001	Three-layer architecture with SFML-free logic and persistence.	Monolithic single class; full MVC; Entity-Component-System (ECS).	Monolith fails NFR-005/007. MVC maps onto the same split but adds a controller layer we do not need. ECS is over-engineered for one grid game. Layering gives testability and clear ownership; cost: a few interfaces to maintain.
ADR-002	Fixed-timestep tick decoupled from render frames; tick interval = difficulty value (× boost factor).	Tick-per-frame; sleep-based loop.	Tick-per-frame ties speed to FPS and breaks FR-002; sleeping harms input latency. Accumulator keeps speed exact and input polled every frame (NFR-002).
ADR-003	Strategy objects for modes and power-up effects.	Conditionals on mode in the engine.	Conditionals spread mode rules across the engine, hurting maintainability; Strategy localises them in C7/C8.
ADR-004	Repository + SecureFileStore; JSON files; write-temp-then-rename.	Binary file; SQLite; direct overwrite.	JSON is human-readable and debuggable and matches SRS Appendix B; SQLite is overkill; direct overwrite risks corruption (SEC-004).
ADR-005	Injectable IClock and IRandom.	Use std::chrono / rand directly.	Deterministic unit tests for timers and random spawns (FR-006, FR-010, FR-012–015).
ADR-006	Single-threaded game logic on the main thread.	Separate logic thread.	Avoids races; workload is tiny (600 cells). SFML audio uses its own thread but shares no game state.

3.6 Technology Stack & Data Stores

Category	Choice
Language / standard	C++17

Category	Choice
Graphics, audio, input	SFML 2.5.1 (Graphics, Audio, Window, System)
Build	CMake; separate library targets per module (engine, modes, persistence, ui) plus test executables
Persistence	JSON via nlohmann/json 3.x; files in a `data/` directory next to the executable (path injectable for tests)
Testing / quality	Catch2, gcov/lcov, ASan/UBSan, cppcheck, GitHub Actions
Design tooling	UML diagrams generated with Graphviz/Matplotlib; editable in draw.io/PlantUML

Data stores. `leaderboard.json` — ≤ 10 entries (§4.3.8); `config.json` — settings; `snakextreme.log` — diagnostics. Each JSON file carries a `version` field, is ≤ 64 KiB by rule (SEC-007) and is retained until manually deleted (SRS §7). In memory: the grid is 30×20; the snake is a `std::deque<Cell>` with a 600-bit occupancy set giving O(1) collision checks.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Corrupt or missing leaderboard/config file	Crash or lost scores	Schema validation, defaults, atomic writes (C13); TC-LB-04, TC-SEC-03/04.
Frame-rate drops on low-end GPUs	Fails NFR-001	Simple 2D drawing, batch sprites in one vertex array, FPS probe in tests; fixed-timestep keeps logic correct at low FPS.
Input latency perceived at slow tick rates	NFR-002 interpretation	Input polled every frame and queued; acknowledgement ≤ 1 frame; open issue OI-1.
Time- and random-dependent logic is hard to test	Flaky tests	IClock/IRandom seams (ADR-005).
Team members' modules diverge	Integration failures in Sprint 6	Interfaces frozen before Sprint 3 (§4.3); CI builds all targets on every PR.
Third-party library version drift (SFML, nlohmann/json)	Build break	Pin versions in CMake; CI uses pinned versions.
Windows rename failure due to locked file	Save fails	Error result, old file preserved, user informed non-fatally (§4.4).

3.8 Traceability to Requirements

Each requirement is allocated to components (Figure 1) and design elements. Test cases are listed in the Test Plan §13 and the combined matrix in SRS Addendum §5.

Req ID	Requirement (short)	Components	Design ref
FR-001	Render grid + snake ≤ 2 s	C1, C2, C4, C6	§4.1, §4.5
FR-002	Tick interval per difficulty (200/120/70 ms)	C6, C7, C15	§4.1, SD-1
FR-003	Direction control; no 180° reversal	C3, C6	§4.1, SD-1
FR-004	Wall collision ends game	C6, C7	§4.1, SD-1
FR-005	Self collision ends game	C6	§4.1, SD-1
FR-006	Food spawns on random free cell	C6, C15	§4.1
FR-007	Growth by exactly one segment	C6	§4.1, SD-1
FR-008	Select mode (Classic/Timed/Obstacle)	C2, C7	§4.5
FR-009	Select difficulty (Easy/Medium/Hard)	C2, C7	§4.5
FR-010	Timed mode ends at 0	C6, C7	§4.1, SD-1

Req ID	Requirement (short)	Components	Design ref
FR-011	Obstacles fatal in Obstacle mode	C6, C7	§4.1
FR-012	Power-up spawn interval ≤ 15 s	C8, C15	SD-3
FR-013	Speed boost 5 s	C6, C8	SD-3
FR-014	×2 multiplier 10 s	C8, C9	SD-3
FR-015	Shrink by one (min length 3)	C6, C8	SD-3
FR-016	Mode-specific score per food	C7, C9	§4.1, SD-1
FR-017	Real-time score on HUD	C2, C4, C9	§4.5
FR-018	Persist Top-10 with all fields	C11, C13	§4.3, SD-2
FR-019	View leaderboard from main menu	C2, C11	§4.3, §4.5
FR-020	Insert only if within Top-10	C11	SD-2
FR-021	Validate on load; default if invalid	C11, C13, C14	SD-2, §4.4
FR-022	Main menu options	C1, C2	§4.5
FR-023	Name entry, max 15 chars	C2, C10	§4.5, SD-2
FR-024	Pause/resume with Esc	C1, C2, C3	§4.5
FR-025	Game Over screen with options	C2	§4.5, SD-2
FR-026	Remap four movement keys	C2, C3, C12	§4.3, §4.5
FR-027	Independent music/SFX volume	C2, C5, C12	§4.5, SD-4
FR-028	Select visual theme	C2, C4, C12	§4.5
FR-029	Persist settings across restarts	C12, C13	SD-4
NFR-001	≥ 30 FPS	C1, C4, C6	§3.5 (ADR-002)
NFR-002	Input latency ≤ 100 ms	C1, C3, C6	§3.5, SD-1
NFR-003	No crash on missing/corrupt files	C11, C12, C13, C14	§4.4, SD-2, SD-4
NFR-004	First game within 3 screens	C1, C2	§4.5
NFR-005	Modular structure with unit tests	All layers	§3.5 (ADR-001)
NFR-006	Builds/runs on Windows 10/11	Build (CMake)	§3.6
NFR-007	Logic testable without SFML context	C6–C10, C15	§3.5 (ADR-001)
SEC-001	Sanitise player name	C2, C10	§3.9, SD-2
SEC-002	No PII beyond display name	C11, C12, C14	§3.9, §4.4
SEC-003	Schema-validate files before use	C11, C12, C13	§3.9, SD-4
SEC-004	Write-temp-then-rename	C13	§3.9, SD-2
SEC-005	File data is data, never code	C11, C12, C13	§3.9
SEC-006	Fixed max buffer for name input	C2, C10	§3.9, §4.3
SEC-007	Refuse files > 64 KiB (new, Addendum)	C13	§3.9

Component allocation by feature owner: Feature 1 Core Engine → C6 · Feature 2 Modes/Power-ups → C7, C8 · Feature 3 Scoring/Leaderboard → C9, C11, C13 · Feature 4 Menu/Settings → C1–C5, C12 (C10 shared with Feature 3).

3.9 Security Architecture

Assets: leaderboard and settings files (integrity, availability); player display name (minimal privacy); the process itself (memory safety). Trust boundaries: TB-1 keyboard/text input → application; TB-2 local file system ↔ application. Nothing else crosses a boundary (no network).

Threat model (STRIDE) adapted to an offline, single-user desktop game:

Threat	Scenario	Control	Req.
Spoofing	Anyone can type any display name; no authentication exists.	Accepted: names are self-declared labels, not identities.	SEC-002
Tampering	User hand-edits leaderboard.json/config.json, or a crash truncates a file.	Schema + range + enum validation; atomic temp-then-rename writes; defaults on failure. Score inflation by editing files is an accepted residual risk (no signing in v1.0).	SEC-003, SEC-004
Repudiation	Disputed high scores.	Not applicable (single-user, local); diagnostic log kept.	—
Information disclosure	Files/logs leaking personal data or paths.	Only display name stored; log uses relative paths and no name or file contents.	SEC-002
Denial of service	Huge, deeply nested or garbage file; over-long name input.	64 KiB size cap before parsing; parser depth limit; fixed 15-char buffer; fallback to defaults.	SEC-003, SEC-006, SEC-007
Elevation of privilege	Crafted strings executed or used as format strings/paths.	File data only displayed as text; no `system/exec/dlopen`; no file-derived format strings; name whitelist blocks path/control characters; runs with normal user rights.	SEC-001, SEC-005

Security controls by component

Component	Security responsibility
C10 InputValidator	Single source of truth for the name whitelist and 15-char cap; used at input time (C2) and again before persistence (C11) — defence in depth.
C13 SecureFileStore	Reads at most 64 KiB; parses JSON with bounded depth; validates schema; writes to `*.tmp`, flushes, closes, renames; removes stale temp files at start-up.
C11 / C12 Repositories	Range/enum checks (score ≥ 0, volume 0–100, enums), sorted/limited entries, default fallback; never propagate raw file strings into commands or format strings.
C14 Logger	No sensitive data; no file contents; size-limited log rotation.
Build & CI	Compiler warnings as errors, cppcheck, ASan/UBSan test runs, banned-API grep (`system`, `popen`, `exec*`, `strcpy`, `sprintf`), pinned dependency versions.

4. Design

4.1 Design Overview

The design follows the layering in §3. The runtime is a single-threaded loop owned by C1. Each render frame: poll events → C3 maps them to actions → active screen handles actions → the gameplay screen calls `GameEngine::update(dt)` → the renderer draws. Movement advances in ticks derived from an accumulator (ADR-002).

Key classes

Class / type	Layer	Purpose
`Cell {x,y}`, `Direction`, `Snake` (deque + occupancy)	Logic	Grid coordinates and snake body; O(1) collision lookups.

Class / type	Layer	Purpose
`GameState`	Logic	Snake, food, obstacles, power-up, score, remaining time, status (Running/Paused/GameOver), mode, difficulty.
`GameEngine` (C6)	Logic	`startSession`, `queueDirection`, `update(dt)`, `pause`, `drainEvents`.
`IModeStrategy` → `ClassicMode`, `TimedMode`, `ObstacleMode`	Logic	`isFatal`, `timeLimit`, `foodPoints`, `placeObstacles`.
`DifficultyProfile`	Logic	Base tick interval: 200/120/70 ms.
`PowerUpManager`, `IPowerUpEffect` → `SpeedEffect`, `MultiplierEffect`, `ShrinkEffect`	Logic	Spawn schedule, pick-up, timed apply/revert.
`ScoreService`	Logic	Score, multiplier, per-mode points.
`LeaderboardEntry`, `Settings`	Logic/Data	Plain data structures matching SRS Appendix B.
`LeaderboardRepository`, `SettingsRepository`, `SecureFileStore`	Persistence	File-backed storage with validation and atomic write.
`ScreenManager`, `IScreen` and 8 screen classes	Presentation	State machine for menus/gameplay (§4.5).
`InputMapper`, `Renderer`, `ThemeProvider`, `AudioService`	Presentation	SFML-facing services.

Baseline design parameters (from SRS Addendum §4)

Parameter	Value	Requirement(s)
Grid	30 × 20 cells; cell = 30 px; window 900 × 600 px	SRS 2.6, 3.1
Initial snake	3 segments, centre of grid, heading right	FR-015, TC-CORE-01
Tick interval	Easy 200 ms · Medium 120 ms · Hard 70 ms	FR-002
Timed Challenge duration	60 s	FR-010
Food points	Classic 10 · Timed 10 · Obstacle 15 (before multiplier)	FR-016
Speed boost	tick interval × 0.6 for 5 s	FR-013
Multiplier	× 2 on food points for 10 s	FR-014
Shrink	−1 segment, floor at 3	FR-015
Power-up spawn gap	random 8–15 s after previous spawn; one item on grid; uncollected item is relocated	FR-012
Obstacle count	8 static cells, ≥ 3 cells from snake head, never on food	FR-011
Name rule	1–15 chars from [A-Za-z0-9 _-], trimmed	FR-023, SEC-001/006
Leaderboard tie rule	Equal score ranks below existing entry; must strictly beat 10th when full	FR-020
Default settings	Arrow keys (WASD alias always on) · music 70 · SFX 70 · theme Classic	FR-026–029
File size cap	leaderboard.json / config.json ≤ 64 KiB	SEC-003, SEC-007
Direction queue	≤ 2 pending inputs, validated against last queued heading	FR-003

Key algorithms and rules

- **Tick:** `acc += dt; while (acc ≥ interval) { acc -= interval; step(); }` where `interval = difficultyBase × effectFactor`. `update` is a no-op while paused or after game over.
- **Step order:** apply next queued direction (drop 180° reversals against the last applied heading) → compute new head → ask ModeManager `isFatal` (wall, obstacle, self; the tail cell counts as free if the

snake is not growing) → on food: grow, score, spawn food; else move → PowerUpManager pick-up check → timers.

- **Direction queue:** at most 2 pending inputs, each validated against the last queued heading, so two quick key presses inside one tick cannot produce an instant reversal (FR-003).
- **Food/power-up/obstacle placement:** collect the list of free cells (≤ 600) and pick uniformly with `IRandom`; if none is free the session ends as “board full”.
- **Effects:** only one active effect; collecting a new power-up first reverts the current one (business rule, SRS 6.5). Speed multiplies the tick interval by 0.6; Multiplier doubles food points; Shrink removes one tail segment but never below length 3.
- **Leaderboard ranking:** sort by score descending, ties keep earlier entries first; a new score qualifies if fewer than 10 entries exist or it strictly exceeds the 10th.

4.2 UML Sequence Diagrams

Four sequence diagrams cover the main runtime flows and the key security/robustness flows.

SD-1 Gameplay tick (FR-001–007, FR-016/017, NFR-002) — a key press is mapped to an action, queued in the engine, and applied at the next tick; collision and food are evaluated; score and HUD are updated.

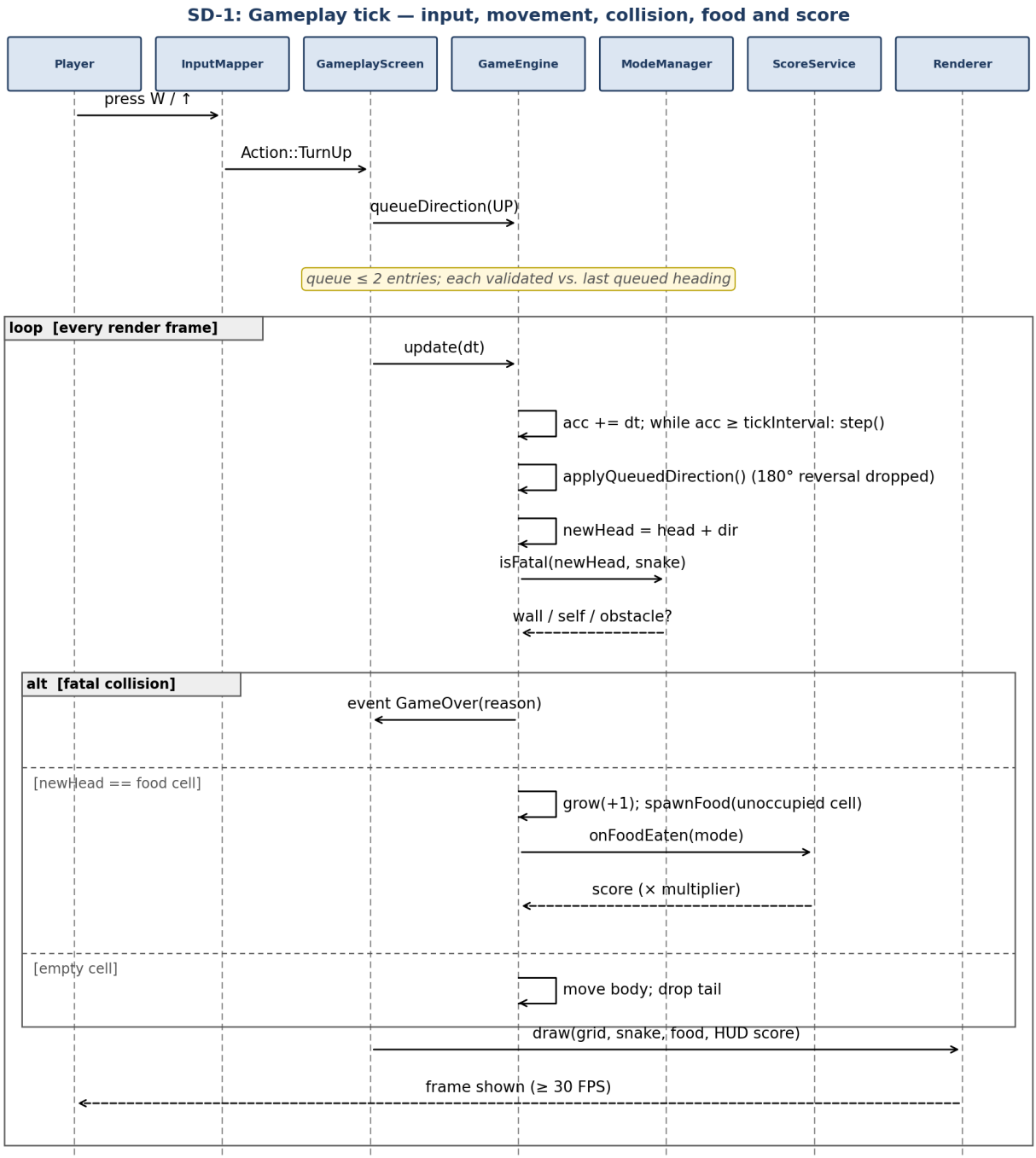


Figure 2 — SD-1: Gameplay tick

SD-2 Game over → save high score (FR-018, FR-020, FR-021, FR-025, SEC-001/003/004) — validated, ranked, atomically persisted.

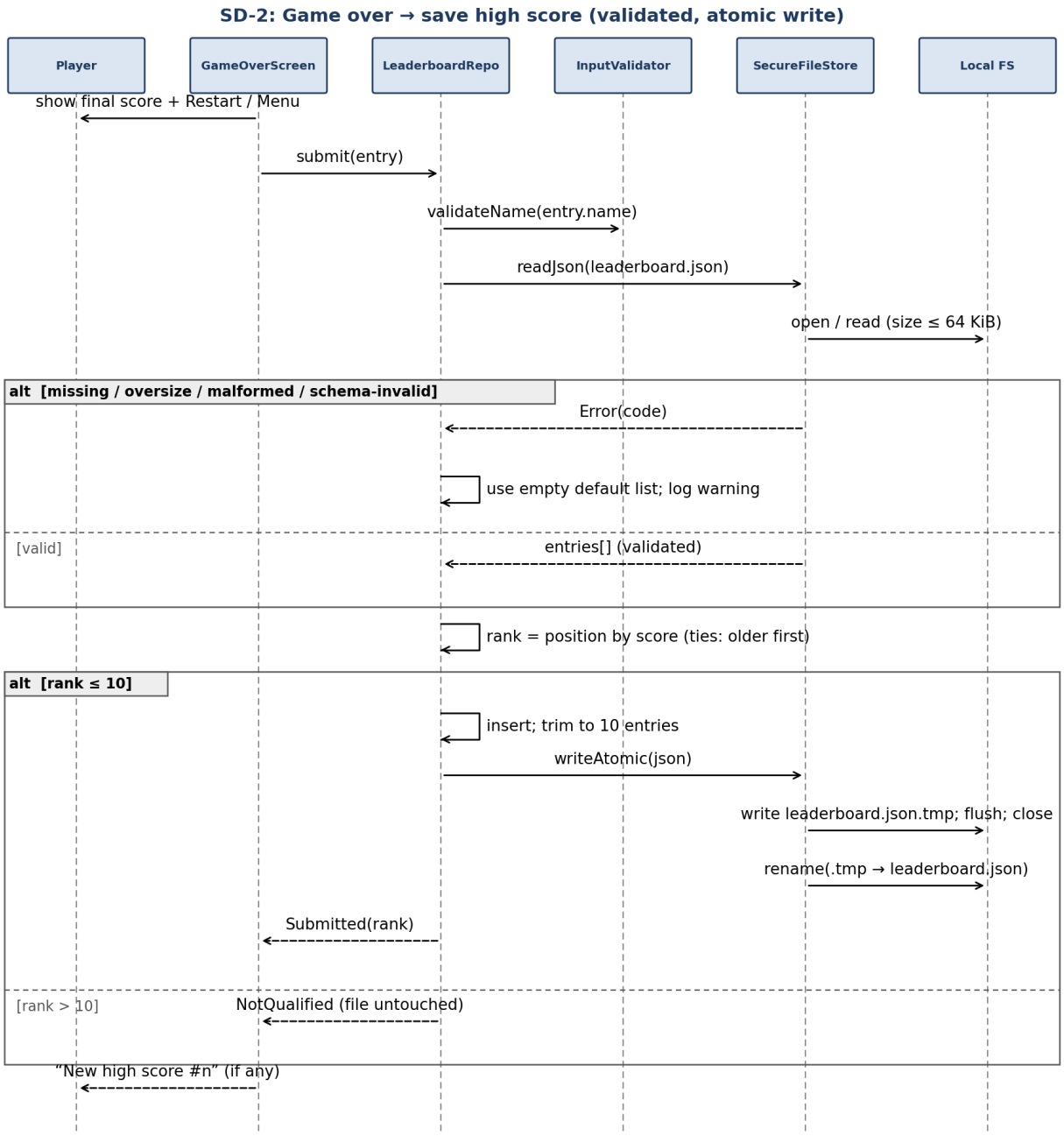


Figure 3 — SD-2: Save high score

SD-3 Collect power-up (FR-012–015, SRS 6.5) — pick-up, replacement of any active effect, timed apply/revert.

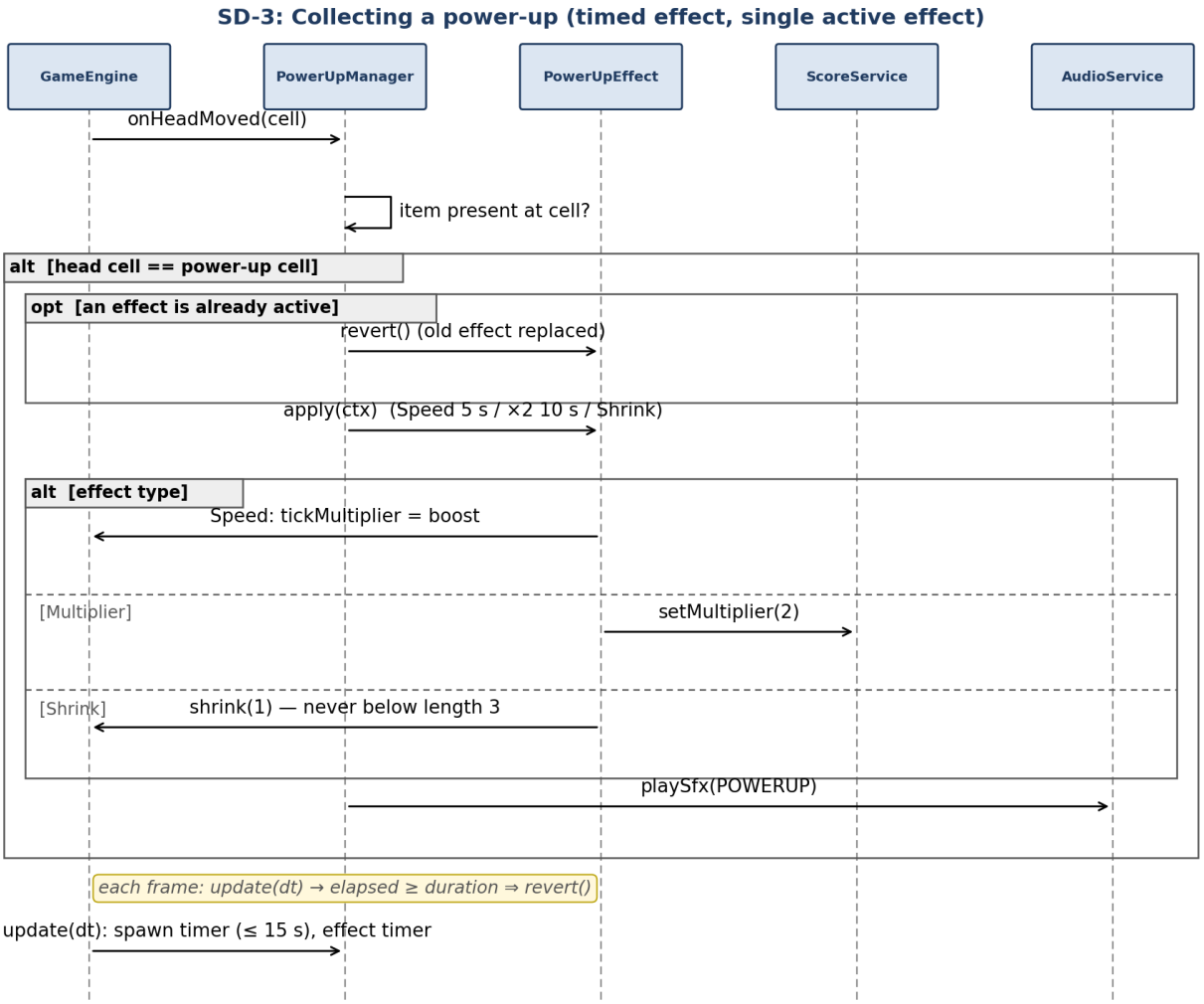


Figure 4 — SD-3: Collect power-up

SD-4 Start-up settings load (FR-027, FR-029, NFR-003, SEC-003) — schema validation with defaults fallback.

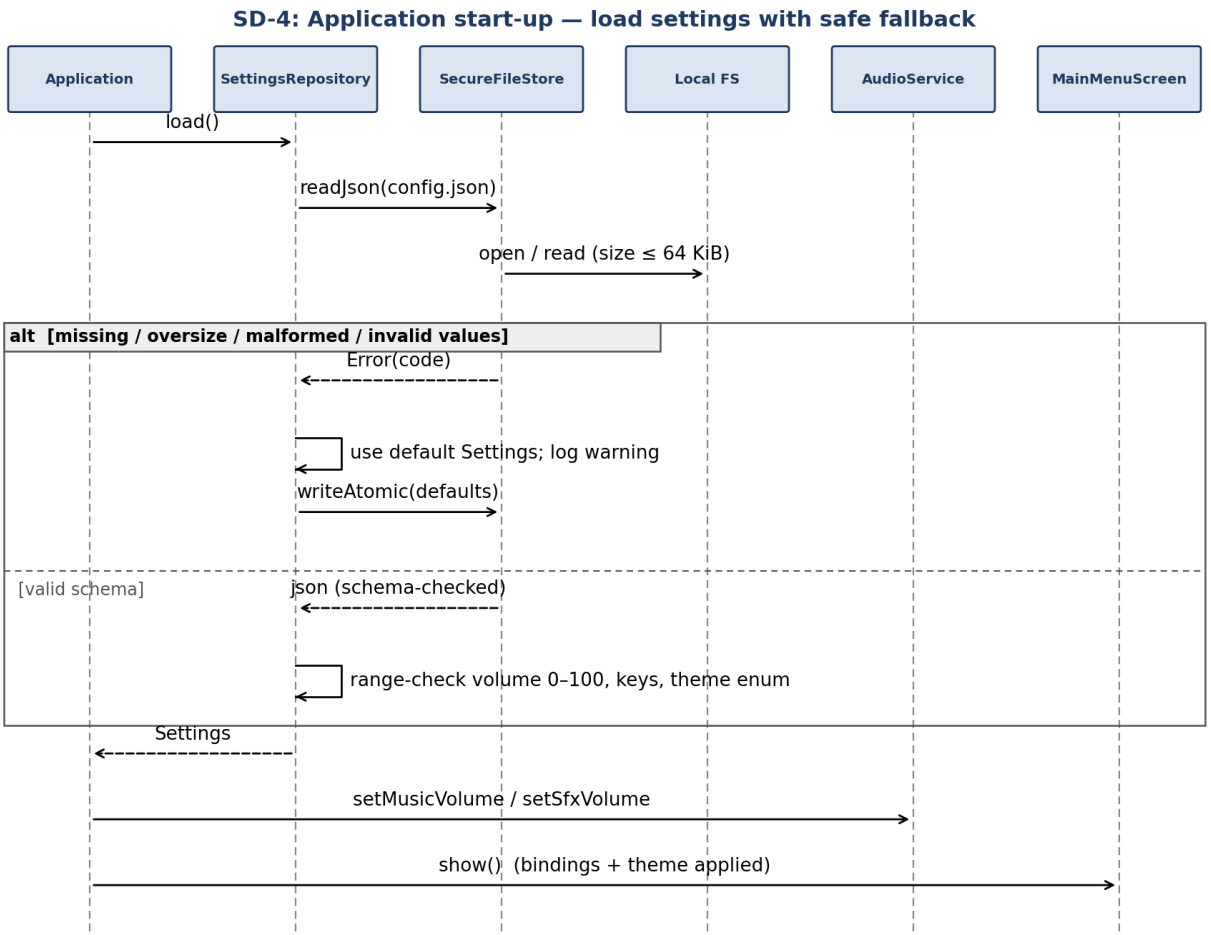


Figure 5 — SD-4: Load settings at start-up

4.3 API Design

SnakeXtreme has no network API; its interfaces are internal C++ interfaces between components. They are frozen before Sprint 3 so owners can develop and test in parallel. All fallible operations return `Result<T>` (value or `Error{code,message}`); nothing throws across a component boundary.

4.3.1 IGameEngine (C6)

Operation	Description	Errors / behaviour on misuse
`void startSession(const SessionConfig&)`	Resets state for `{mode, difficulty}`; snake length 3 at centre, heading right; places food and obstacles.	Invalid enum → `E_BAD_CONFIG` (programming error, asserted in debug).
`void queueDirection(Direction)`	Queues a turn (max 2). 180° reversals are silently dropped.	None; ignored when paused/over.
`void update(Duration dt)`	Advances time; runs 0..n ticks; updates timers and effects.	No-op when paused or game over.
`void pause(bool)`	Freezes/unfreezes all timers.	—
`const GameState& state() const`	Read-only snapshot for the renderer/HUD.	—
`std::vector<GameEvent> drainEvents()`	Events: `FoodEaten`, `PowerUpCollected`, `GameOver{WALL|SELF|OBSTACLE|TIME_UP|BOARD_FULL}`.	—

4.3.2 ILeaderboardRepository (C11)

Operation	Description	Errors
`Result<std::vector<LeaderboardEntry>> load()`	Reads and validates `leaderboard.json`.	`E_FILE_MISSING`, `E_FILE_TOO_LARGE`, `E_PARSE`, `E_SCHEMA` → caller uses empty list.
`bool qualifies(int score) const`	True if fewer than 10 entries or score > 10th.	—
`Result<SubmitOutcome> submit(const LeaderboardEntry&)`	Validates name, ranks, inserts, trims to 10, writes atomically. Returns `{Submitted,rank}` or `NotQualified`.	`E_INVALID_NAME`, `E_WRITE_FAILED` (old file kept).

4.3.3 ISettingsRepository (C12) and IInputValidator (C10)

Operation	Description	Errors
`Result<Settings> load()`	Loads/validates `config.json`; range-checks volumes 0–100, key codes and theme enum.	`E_FILE_MISSING`, `E_FILE_TOO_LARGE`, `E_PARSE`, `E_SCHEMA` → `Settings::defaults()`.
`Result<void> save(const Settings&)`	Atomic write of `config.json`.	`E_WRITE_FAILED`.
`ValidationResult validateName(std::string_view)`	Trims and checks 1–15 chars from [A-Za-z0-9 _-].	`TooShort`, `TooLong`, `BadChar{index}`.
`bool isAllowedChar(char32_t)`	Used per keystroke by the name-entry field.	—

4.3.4 ISecureFileStore (C13), IPowerUpManager (C8), IModeStrategy (C7)

Interface	Operations	Notes
ISecureFileStore	`Result<json> readJson(path, maxBytes=65536)`; `Result<void> writeAtomic(path, const json&)`; `void removeStaleTemp(path)`	Bounded read; parser depth-limited; write = temp → flush → close → rename.
IPowerUpManager	`void update(Duration)`; `void onHeadMoved(Cell)`; `optional<ActiveEffect> active() const`; `optional<PowerUp> onGrid() const`	Owns spawn timer (8–15 s) and single active effect.
IModeStrategy	`bool isFatal(Cell, const Snake&, const Obstacles&) const`; `optional<Duration> timeLimit() const`; `int foodPoints() const`; `void placeObstacles(IRandom&, const Snake&, Obstacles&)`	`ClassicMode` (no limit, no obstacles), `TimedMode` (60 s), `ObstacleMode` (8 obstacles).

4.3.5 File schemas (validated by C13 before use)

`leaderboard.json` — field limits from SRS Appendix B:

```
{ "version": 1,
  "entries": [
    { "player_name": "Ayushi",        // 1-15 chars, [A-Za-z0-9 _-]
      "score": 240,                  // integer 0 .. 2147483647
      "game_mode": "Classic",        // Classic | Timed | Obstacle
      "difficulty": "Hard",          // Easy | Medium | Hard
      "date_achieved": "2026-09-28" } // valid YYYY-MM-DD
  ] }                                // at most 10 entries, score descending
```

`config.json`:

```
{ "version": 1,
  "key_up": 73, "key_down": 74,       // SFML key codes (integers), all four distinct
  "key_left": 71, "key_right": 72,
  "music_volume": 70, "sfx_volume": 70, // integers 0 .. 100
  "theme": "Classic" }                  // Classic | Neon | Retro
```

Validation rules: file ≤ 64 KiB; root must be an object with the expected keys and types; unknown keys are ignored; any violation → `E_SCHEMA` and defaults.

4.3.6 Error codes

Code	Meaning	Raised by
E_FILE_MISSING	File does not exist (first run)	C13
E_FILE_TOO_LARGE	File exceeds 64 KiB	C13
E_PARSE	Not valid JSON / depth limit exceeded	C13
E_SCHEMA	Types, ranges or enums invalid	C11, C12
E_INVALID_NAME	Name violates whitelist/length	C10, C11
E_WRITE_FAILED	Temp write, flush or rename failed	C13
E_BAD_CONFIG	Invalid session configuration (programming error)	C6

4.4 Error Handling, Logging & Monitoring

Situation	Detection	Handling	User-visible result	Req.
Leaderboard/config missing (first run)	`E_FILE_MISSING`	Use defaults; create file on first save.	None (empty leaderboard)	NFR-003
Corrupt/oversize/schema-invalid file	`E_PARSE`, `E_FILE_TOO_LARGE`, `E_SCHEMA`	Log warning; use defaults; file replaced on next valid save.	Empty leaderboard / default settings; no crash	FR-021, SEC-003/007
Save fails (locked file, disk full)	`E_WRITE_FAILED`	Old file kept; temp removed; retry once.	Non-fatal notice “Could not save score/settings”	SEC-004
Invalid name typed	`isAllowedChar` / `validateName`	Character ignored or submission blocked.	Inline message below the field	FR-023, SEC-001/006
Duplicate key binding	Settings screen check	Reject binding.	Inline message	FR-026
Asset (font/sound/texture) missing	SFML load failure	Fall back to built-in shapes/system font; log.	Game still playable (silent if audio missing)	NFR-003
Unexpected exception	Top-level `try/catch` in `main`	Log; try to save settings; exit cleanly.	Error dialog, no data corruption	NFR-003

Logging. `snakextreme.log` in `data/`; levels ERROR/WARN/INFO/DEBUG; format `2026-09-28T10:15:03 [WARN] SecureFileStore: config.json schema invalid (E_SCHEMA); defaults loaded`. The log never contains player names, file contents or absolute paths (SEC-002); it is capped at 256 KiB with one rotated backup.

Monitoring. Debug builds show an FPS/tick overlay (F3) and, with `-DSNAKE_PROBE`, write FPS windows and input-latency samples used by TC-PERF-01/02. CI reports test pass rate, coverage, and static-analysis findings.

4.5 UX Design

The window is 900×600 px; the 30×20 grid uses 30-px cells. The HUD (score, mode, remaining time in Timed mode, active power-up with seconds left) is drawn as a translucent overlay so the grid keeps its full size. All screens share one convention: Enter/click confirms, Esc goes back or pauses, arrows/WASD navigate.

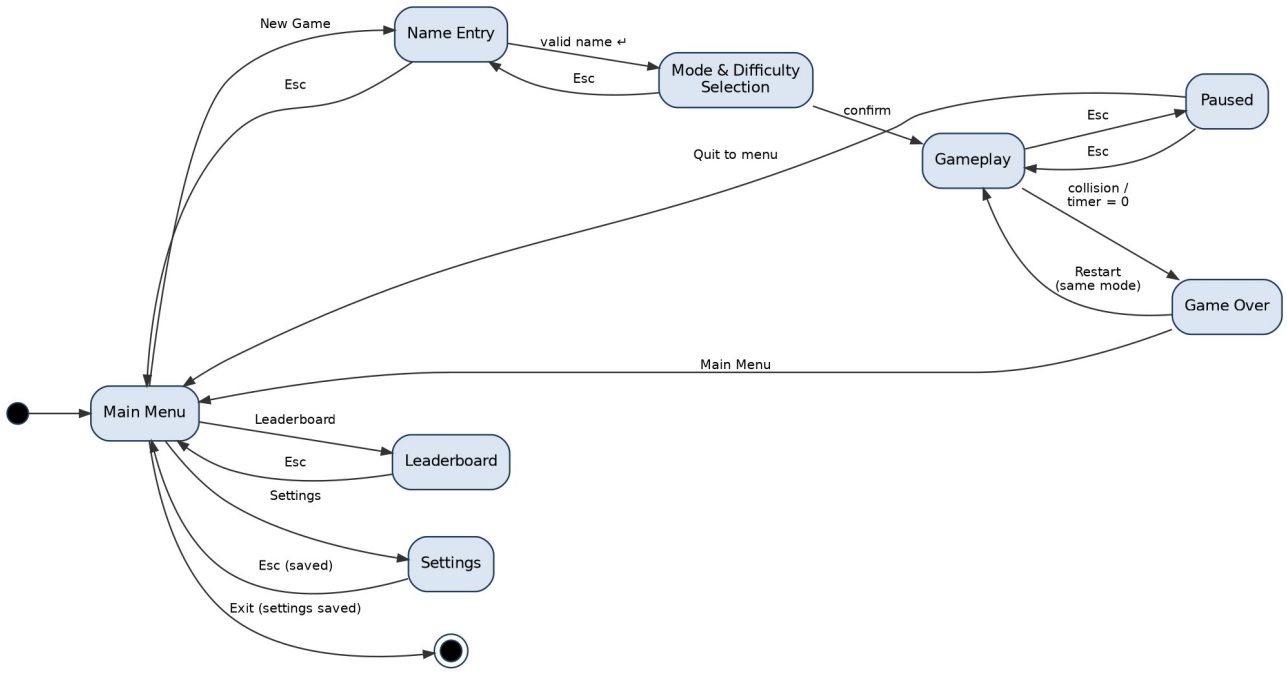


Figure 6 — Screen navigation state machine

Screen	Purpose	Controls	Feedback / errors
Main Menu	New Game, Leaderboard, Settings, Exit (FR-022)	↑↓/WASD, Enter, mouse	Key hints in footer (NFR-004)
Name Entry	Enter display name ≤ 15 chars (FR-023)	Typing, Backspace, Enter, Esc	Inline text for empty/invalid characters
Mode & Difficulty	Choose mode and difficulty (FR-008/009)	← → / ↑↓, Enter, Esc	Highlight; short description of each mode
Gameplay	Play; HUD	Steering keys, Esc = pause	Power-up icon + countdown; Timed countdown
Pause overlay	Resume / quit to menu (FR-024)	Esc, Enter	Game frozen behind overlay
Game Over	Final score; Restart / Main Menu (FR-025)	← →, Enter	“New high score #n” when applicable
Leaderboard	Top-10 list (FR-019)	Esc	“No scores yet” when empty
Settings	Keys, music/SFX volume, theme (FR-026–029)	↑↓, ← →, Enter, Esc	Inline message for duplicate keys; theme preview

Accessibility and learnability. Food, obstacles and power-ups differ by shape as well as colour; each theme keeps high contrast; keys are remappable; the first game is reachable in three screens (Main Menu → Name Entry → Mode & Difficulty).

4.6 Open Issues & Next Steps

ID	Open issue	Proposed resolution
OI-1	NFR-002 “input-to-response ≤ 100 ms” vs. tick intervals up to 200 ms: the visible turn happens at the next tick.	Measure acknowledgement latency (engine registers the turn) ≤ 100 ms; record visible turn. Ask product owner to confirm.
OI-2	FR-005 does not say whether the vacating tail cell is safe.	Treat it as free when the snake is not growing (standard rule); confirm.
OI-3	Score per food, Timed duration, speed-boost factor, obstacle count are unspecified.	Baselines in SRS Addendum §4; confirm or adjust `GameConfig`.
OI-4	NFR-001 “target hardware profile” undefined.	Defined in STP §6.

ID	Open issue	Proposed resolution
OI-5	File name mismatch: Synopsis `leaderboard.dat` vs SRS `leaderboard.json`.	Use JSON; update Synopsis.
OI-6	Score tampering by editing the JSON is not prevented.	Accepted for v1.0; future: HMAC/checksum.
OI-7	Audio/graphics asset licensing.	Use original or openly licensed assets; record sources.

Next steps: freeze the interfaces in §4.3 (before Sprint 3); create CMake targets per module and the Catch2 skeleton with Fake Clock/RNG; implement per the sprint plan, adding traceability (code file refs) to the RTM; run the Test Plan cases each sprint; future enhancements: Linux build, gamepad support, signed leaderboard, additional modes.

5. Appendices

5.1 Glossary — *Tick*: one snake movement step. *Render frame*: one window redraw. *ADR*: architecture decision record. *Repository*: interface hiding persistence. *Result<T>*: value-or-error type. *Atomic write*: write to a temp file, then rename over the target. *STRIDE*: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. *HUD*: heads-up display. *SFML*: Simple and Fast Multimedia Library.

5.2 References — IEEE 42010 (architecture description); IEEE 1016 (software design descriptions); NIST SP 800-160; OWASP ASVS; SEI CERT C++ Coding Standard; SFML 2.5 documentation; nlohmann/json documentation; SRS v1.0, SRS Addendum v1.1, STP v1.0, Team 16 Synopsis v1.0.

5.3 Tools — Graphviz, Matplotlib (diagrams in this document); draw.io/PlantUML (recommended for later edits); CMake, Catch2, cppcheck, GitHub Actions.

PES UNIVERSITY, BANGALORE — Department of Computer Science and Engineering

Software Test Plan (STP)

SnakeXtreme – Multi-Mode Snake Game with Power-ups & Leaderboard

Project	SnakeXtreme (Team 16)
Version	1.0
Authors	Team 16 — Ayushi Srivastava (QA Lead), B V Tejas, Bharath Hegde, C Dinesh Kumar
Date	28-09-2026
Status	Draft — Part-1 submission (includes test-case update)
Format	IEEE 829 / ISO/IEC/IEEE 29119-3 style, following the course STP template

Revision History

Version	Date	Author	Change summary
1.0	28-Sep-2026	Team 16	Initial test plan with 44 test cases (Appendix A), traceability to SRS v1.0 + Addendum v1.1.

1. Introduction

Purpose: This document defines the test plan for SnakeXtreme v1.0: objectives, scope, strategy, environment, schedule, responsibilities, and the test cases that verify every requirement in the SRS. It is the validation baseline for the Implementation & Testing deliverable.

Scope: Testing covers core gameplay (movement, collision, food, growth), game modes/difficulty/obstacles, power-ups, scoring, leaderboard and settings persistence, menu/UI navigation, and the non-functional and security requirements (performance, reliability, usability, maintainability, portability, testability, input/file security). Third-party library internals and operating-system behaviour are excluded (see §4).

References:

- SnakeXtreme SRS v1.0 (31-Aug-2026) and SRS Addendum v1.1 (28-Sep-2026)
- SnakeXtreme Software Architecture & Design Specification (SAD) v1.0
- Team 16 Synopsis v1.0
- IEEE 829-2008 / ISO/IEC/IEEE 29119-3; OWASP ASVS (input validation, file handling)

Definitions: SRS (Software Requirements Specification); SAD (Software Architecture & Design); RTM (Requirements Traceability Matrix); FR/NFR/SEC (functional / non-functional / security requirement); TC (test case); BVA (boundary value analysis); Fake Clock (test double that replaces wall-clock time so timers can be advanced instantly); ASan/UBSan (AddressSanitizer / UndefinedBehaviorSanitizer); FPS (frames per second).

2. Test Items

Items are the SAD components (see SAD §3.4). Build under test: SnakeXtreme v1.0 release candidate built from the `main` branch.

Test item	SAD component	Feature owner
Core Gameplay Engine (movement, collision, food, growth)	C6 GameEngine	Ayushi Srivastava
Game modes, difficulty scaling, obstacles	C7 ModeManager	B V Tejas

Test item	SAD component	Feature owner
Power-up system (Speed, Multiplier, Shrink)	C8 PowerUpManager	B V Tejas
Score calculation	C9 ScoreService	Bharath Hegde
Player-name validation	C10 InputValidator	Bharath Hegde
Leaderboard persistence and ranking	C11 LeaderboardRepository	Bharath Hegde
Secure JSON file store (schema check, size cap, atomic write)	C13 SecureFileStore	Bharath Hegde
Settings persistence	C12 SettingsRepository	C Dinesh Kumar
Menus, screens, input mapping, rendering, audio	C1–C5	C Dinesh Kumar
Build system and CI (portability, headless testability)	CMake, GitHub Actions	B V Tejas

3. Features to be Tested

Every requirement in SRS v1.0/Addendum v1.1 is in scope: 29 FRs, 7 NFRs and 7 security requirements (43 total). Features are grouped as follows; the requirement-to-test mapping is in §13.

Feature group	Requirements	Test cases
Core gameplay: render, tick speed, steering, wall/self collision, food, growth	FR-001 – FR-007	TC-CORE-01 … 08
Modes, difficulty, timer, obstacles	FR-008 – FR-011	TC-MODE-01 … 04
Power-ups and business rule “one active effect”	FR-012 – FR-015, SRS §6.5	TC-PWR-01 … 05
Scoring and HUD score	FR-016, FR-017	TC-SCORE-01, 02
Leaderboard persistence, ranking, validation	FR-018 – FR-021	TC-LB-01 … 04
Menus, name entry, pause, game over	FR-022 – FR-025	TC-UI-01 … 04
Settings: keys, volume, theme, persistence	FR-026 – FR-029	TC-SET-01 … 04
Performance	NFR-001, NFR-002	TC-PERF-01, 02
Reliability, usability, maintainability, portability, testability	NFR-003 – NFR-007	TC-REL-01, TC-USE-01, TC-MNT-01, TC-PORT-01, TC-TEST-01
Security: sanitisation, PII, schema/size validation, safe write, untrusted data, buffer limits	SEC-001 – SEC-007	TC-SEC-01 … 06

4. Features Not to be Tested

- Internal correctness of SFML 2.5+ and nlohmann/json (third-party; only our usage is tested).
- GPU drivers, OpenGL implementation and audio hardware (environmental; observed only through smoke tests).
- Linux/macOS builds — stretch goal per SRS 2.5; a Linux build is run for headless logic tests only and is not a pass/fail criterion for release.
- Multiplayer, network, cloud leaderboard, AI opponent, monetisation, mobile/web (out of scope, SRS 2.3).
- Touch or gamepad input; localisation (English only, SRS §7).
- Concurrent multi-instance writes to the leaderboard (out of scope, SRS 2.7).
- Subjective audio/graphics quality and art asset licensing (checked in review, not in test cases).

5. Test Approach / Strategy

Levels

Level	Focus	Tooling	Executed by
Unit	C6–C10, C15, and repositories against fake Clock/RNG and temp directories. Runs headless (NFR-007).	Catch2, gcov/lcov, ASan/UBSan	Each owner for own module; QA Lead reviews
Integration	Engine ↔ ModeManager ↔ PowerUpManager ↔ ScoreService; repositories ↔ SecureFileStore ↔ file system; settings ↔ UI logic.	Catch2 with temp data dir, fixtures	Owners; run in CI on every PR
System	Full application on Windows: menus → gameplay → game over → leaderboard/settings; timing and performance probes.	Manual checklists, in-app FPS/latency probe	All members; QA Lead consolidates
Acceptance	Demonstration of all High-priority requirements to the product owner.	Traceability matrix, demo script	Team + course instructor

Types and techniques

- **Functional** black-box tests derived from each FR; **negative** tests for invalid input and files; **boundary** tests (BVA) at 0/1/15/16 characters, 9.9/10.0 s, tie scores, length 3.
- **State-transition** testing for screen navigation and pause/resume (SAD §4.5); **equivalence partitioning** for modes × difficulties.
- **Regression:** the full automated suite runs on each push/PR (GitHub Actions); manual regression checklist before each sprint demo.
- **Performance** (NFR-001/002) using an in-app probe; **usability** walkthrough (NFR-004); **portability/build** verification (NFR-006); **static analysis** (cppcheck; NFR-005).
- **Determinism:** all timing logic uses an injected Fake Clock and all randomness a seeded RNG, so time- and chance-dependent tests are repeatable.

Entry criteria

- Build compiles cleanly (CMake, C++17) with warnings enabled; smoke test passes.
- Fixtures and test data prepared (see §6); CI pipeline green on the branch.
- For system tests: features for the sprint merged and peer-reviewed.

Exit criteria

- 100 % of planned test cases executed; 100 % of High-priority test cases passed.
- 0 open Critical/High defects; Medium defects have an agreed workaround or fix plan.
- Requirement coverage 100 % (all 43 requirements have ≥ 1 passing test); line coverage of C6–C11 ≥ 80 %.
- No sanitiser (ASan/UBSan) or cppcheck error-level findings.

Defect severity

Severity	Definition	Example
Critical	Crash, data loss or file corruption	Crash on corrupt config; partial leaderboard file
High	Requirement not met; no workaround	Wrong tick speed; reversal kills snake
Medium	Requirement partly met; workaround exists	Volume slider off by one; theme not previewed
Low	Cosmetic / minor	Alignment, wording

5.1 Security Validation

Security validation verifies the security objectives SO-1…SO-4 and requirements SEC-001…SEC-007 (SRS Addendum §2). The attack surface is the name field and the two local JSON files, so validation is concentrated on input handling, file parsing and file writing.

Validation activity	Method / tool	Covers	Test case
Name-field injection & sanitisation	Negative tests with delimiter, control, path, Unicode and JSON-like inputs	SEC-001, SO-2	TC-SEC-01
Privacy review	Inspect files/log for PII and absolute paths	SEC-002, SO-3	TC-SEC-02
Schema and size fuzzing (“fuzz-lite”)	25 malformed fixtures per file, incl. 64 KiB+1 and deeply nested JSON; run under ASan/UBSan	SEC-003, SEC-007, SO-1/4	TC-SEC-03, TC-LB-04, TC-REL-01
Fault injection on save	Test hook aborts process at 4 points of write-temp-then-rename; locked-target case	SEC-004, SO-1	TC-SEC-04
Untrusted-data / unsafe-API review	Hostile-string load test; CI grep for system/popen/exec*/dlopen and format-string misuse; cppcheck	SEC-005, SO-2	TC-SEC-05
Buffer-overflow resistance	10 000-character injection; ASan/UBSan build; code review of fixed-capacity buffers	SEC-006, SO-4	TC-SEC-06
Secure-coding gate	Compiler warnings as errors, cppcheck, no raw C-string APIs (strcpy/sprintf) in code review checklist	SO-2, SO-4	TC-MNT-01

Not applicable (documented, not tested): authentication, session management, TLS and network attacks — the product has no accounts or network interface (SRS 3.3). Tamper-proofing of scores against a determined local user is an accepted residual risk (SAD §3.9).

6. Test Environment

Hardware (reference profile)	Windows 10/11 x64 PC; dual-core CPU ≥ 2.0 GHz; 4 GB RAM; integrated GPU with OpenGL 3.3; display ≥ 800×600; keyboard, mouse, speakers/headphones. Also used for NFR-001.
Software	SnakeXtreme v1.0 RC; C++17 (MSVC 2022 or MinGW-w64 GCC 13); CMake ≥ 3.20; SFML 2.5.1; nlohmann/json 3.11; Ubuntu 22.04 CI image for headless logic tests.
Tools	Catch2 v3 (unit/integration), gcov/lcov (coverage), ASan/UBSan, cppcheck, GitHub Actions (CI), GitHub Issues/Projects or Jira (defects), in-app FPS/latency probe (`-DSNAKE_PROBE`), Git.
Test data	`fixtures/`: leaderboard_10.json, leaderboard_3.json, leaderboard_empty.json, 25 malformed leaderboard and 25 malformed config fixtures, oversize 64 KiB+1 file, hostile-string files, name-input corpus, seeded RNG values, Fake Clock scripts. Each test uses a fresh temporary data directory.

7. Test Schedule

The Synopsis fixes six sprints; calendar dates for sprints 2–6 are set by the sprint planning board. Milestones are therefore expressed per sprint, with dates given where already fixed.

Milestone	Timing	Owner
Test plan and test-case design (this document)	28-Sep-2026	QA Lead + all
Test environment: CMake test targets, Catch2, fixtures, CI (GitHub Actions)	Sprint 3	B V Tejas

Milestone	Timing	Owner
Unit tests for engine/modes written alongside features	Sprints 2–4	Ayushi, B V Tejas
Persistence and security tests (TC-LB, TC-SEC, TC-REL)	Sprints 4–5	Bharath
UI/settings system tests and usability walkthrough	Sprint 5	C Dinesh Kumar
Full execution: system, performance, regression	Sprint 6	All
Acceptance / UAT with product owner; Test Summary Report	Final demo	Team + instructor

8. Test Deliverables

- Software Test Plan (this document) and Appendix A test-case specifications
- Automated test code (Catch2) and scripts, fixtures and test data
- Manual test checklists and execution logs (results per TC, per build)
- Defect reports (GitHub Issues/Jira) and coverage/static-analysis reports
- Requirements Traceability Matrix (§13) kept in sync with the SRS
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Ayushi Srivastava	Owns this plan; reviews all test cases; consolidates results; owns TC-CORE tests and Fake Clock/RNG seams
Test Engineer / CI	B V Tejas	CI pipeline, TC-MODE, TC-PWR, TC-MNT, TC-PORT, TC-TEST
Test Engineer / Data	Bharath Hegde	Fixtures and fuzz-lite corpus; TC-SCORE, TC-LB, TC-SEC
Test Engineer / UI	C Dinesh Kumar	TC-UI, TC-SET, TC-PERF, TC-USE; rendering and probe tooling
Developer	Each feature owner	Fix defects in own module; triage within one sprint day
Product Owner	Course Instructor	Approve results and sign off acceptance

10. Risks and Mitigation

Risk	Mitigation
Time-dependent tests are flaky (power-up timers, Timed mode)	Inject Fake Clock; tests never sleep; only system timing tests use real time, with ±10 % tolerance.
Random behaviour (food, obstacles, power-ups) makes results unrepeatable	Seeded RNG in tests; assert invariants (free cell, spacing) rather than fixed positions.
Rendering and audio cannot be fully automated	Keep logic separate from SFML (NFR-007); manual checklists with screenshots for UI/theme/audio.
FPS/latency results vary by machine	Define reference profile; 3 runs; report min/median; use the same machine for regression.
Rename may fail on Windows if antivirus locks the target	SecureFileStore returns an error and keeps the old file; covered by TC-SEC-04.
Limited team time near submission	Owner-based test authoring per sprint; CI executes automated tests on every PR.
Spec ambiguities (e.g. NFR-002 interpretation, tie rules)	Baselines recorded in SRS Addendum §4 and SAD OI list; confirm with product owner early.

11. Assumptions & Dependencies

- Parameter baselines in SRS Addendum §4 (score values, Timed 60 s, speed-boost factor, obstacle count) are accepted; expected results in Appendix A depend on them.
- SFML 2.5.1 and nlohmann/json are available in CI and on test machines.
- Test machines are single-user; only one app instance runs at a time.
- Volunteer testers for the usability test are available in Sprint 5–6.

12. Suspension & Resumption Criteria

Suspend system testing if: the build fails to launch or crashes within 10 seconds of start on the reference machine; a Critical defect blocks more than 30 % of remaining test cases; the CI/test environment is unavailable for more than 4 hours.

Resume when: a fixed build passes the smoke test (launch → play one Classic game → game over → exit); blocking defects are verified closed; the environment is restored. Affected test cases are re-run in full.

13. Test Case Management & Traceability

Each test case has a unique ID `TC-<AREA>-nn`, references the SRS requirement(s) it verifies, and is specified in Appendix A. Results are recorded per build in the execution log; defects are linked to the TC ID. The RTM below is the SRS-to-test view; the SAD §3.8 gives the SRS-to-component view, and SRS Addendum §5 the combined matrix.

Summary: 44 test cases for 43 requirements (29 FR, 7 NFR, 7 SEC): CORE 8, MODE 4, PWR 5, SCORE 2, LB 4, UI 4, SET 4, PERF 2, REL 1, USE 1, MNT 1, PORT 1, TEST 1, SEC 6. 32 are primarily automated; the rest are manual or mixed. Every requirement is covered by at least one test case; 30 test cases are High priority.

Req ID	Requirement (short)	Test case(s)	Level / type
FR-001	Render grid + snake ≤ 2 s	TC-CORE-01	System
FR-002	Tick interval per difficulty (200/120/70 ms)	TC-CORE-02	Unit + System
FR-003	Direction control; no 180° reversal	TC-CORE-03, TC-CORE-04	Unit + System; Unit
FR-004	Wall collision ends game	TC-CORE-05	Unit
FR-005	Self collision ends game	TC-CORE-06	Unit
FR-006	Food spawns on random free cell	TC-CORE-07	Unit
FR-007	Growth by exactly one segment	TC-CORE-08	Unit
FR-008	Select mode (Classic/Timed/Obstacle)	TC-MODE-01	System
FR-009	Select difficulty (Easy/Medium/Hard)	TC-MODE-02	Integration
FR-010	Timed mode ends at 0	TC-MODE-03	Unit + System
FR-011	Obstacles fatal in Obstacle mode	TC-MODE-04	Unit + System
FR-012	Power-up spawn interval ≤ 15 s	TC-PWR-01	Unit
FR-013	Speed boost 5 s	TC-PWR-02, TC-PWR-05	Unit + System; Unit
FR-014	×2 multiplier 10 s	TC-PWR-03, TC-PWR-05	Unit
FR-015	Shrink by one (min length 3)	TC-PWR-04	Unit
FR-016	Mode-specific score per food	TC-SCORE-01	Unit
FR-017	Real-time score on HUD	TC-SCORE-02	System

Req ID	Requirement (short)	Test case(s)	Level / type
FR-018	Persist Top-10 with all fields	TC-LB-01	Integration
FR-019	View leaderboard from main menu	TC-LB-02	System
FR-020	Insert only if within Top-10	TC-LB-03	Unit
FR-021	Validate on load; default if invalid	TC-LB-04	Integration
FR-022	Main menu options	TC-UI-01	System
FR-023	Name entry, max 15 chars	TC-UI-02	System
FR-024	Pause/resume with Esc	TC-UI-03	System
FR-025	Game Over screen with options	TC-UI-04	System
FR-026	Remap four movement keys	TC-SET-01	System
FR-027	Independent music/SFX volume	TC-SET-02	System
FR-028	Select visual theme	TC-SET-03	System
FR-029	Persist settings across restarts	TC-SET-04	Integration
NFR-001	≥ 30 FPS	TC-PERF-01	Performance
NFR-002	Input latency ≤ 100 ms	TC-PERF-02	Performance
NFR-003	No crash on missing/corrupt files	TC-REL-01, TC-LB-04	Integration
NFR-004	First game within 3 screens	TC-USE-01	Usability
NFR-005	Modular structure with unit tests	TC-MNT-01	Static / review
NFR-006	Builds/runs on Windows 10/11	TC-PORT-01	Build verification
NFR-007	Logic testable without SFML context	TC-TEST-01	Unit / CI
SEC-001	Sanitise player name	TC-SEC-01	Unit + System
SEC-002	No PII beyond display name	TC-SEC-02	Inspection
SEC-003	Schema-validate files before use	TC-SEC-03	Integration
SEC-004	Write-temp-then-rename	TC-SEC-04	Integration
SEC-005	File data is data, never code	TC-SEC-05	Review + test
SEC-006	Fixed max buffer for name input	TC-SEC-06	Unit + System
SEC-007	Refuse files > 64 KiB (new, Addendum)	TC-SEC-03	Integration

14. Test Metrics & Reporting

- % of test cases executed and % passed/failed (per build and per requirement group)
- Requirement coverage (target 100 %) and line coverage of C6–C11 (target ≥ 80 %)
- Defect density (defects per module per sprint), defect aging (days open), defects by severity
- Performance results: min/avg/median FPS, latency percentiles (NFR-001/002)
- Reports: CI status on every push; weekly execution summary; final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead	Ayushi Srivastava	
Dev Lead	B V Tejas	

Role	Name	Signature / Date
Product Owner	Course Instructor	

Appendix A — Test Case Specifications

44 test cases. Priority: High/Medium/Low. “Auto” = Catch2/CI or scripted probe; “Manual” = executed against the checklist. Expected values use the baselines in SRS Addendum §4. Owner: AS = Ayushi, BT = B V Tejas, BH = Bharath, DK = Dinesh.

ID	Title	Req.	Level / type / mode	Pri.	Preconditions & test data	Steps	Expected result
TC-CORE-01	Grid and snake render on new game	FR-001	System Functional Manual+Auto · AS	H	App built; name entered; Classic / Medium selected.	1. Confirm mode/difficulty; start timestamp logged. 2. Log timestamp of first frame containing the 30×20 grid and snake. 3. Repeat for 10 launches.	Delta ≤ 2000 ms in all 10 runs; snake has 3 segments at grid centre heading right.
TC-CORE-02	Tick interval per difficulty	FR-002	Unit + System Functional / timing Auto · AS	H	Fake Clock (unit); release build (system).	1. Unit: advance fake clock in 1 ms steps; record step instants for Easy, Medium, Hard. 2. System: log 50 consecutive tick timestamps per difficulty.	Unit: steps exactly at 200 / 120 / 70 ms. System: mean interval within ±10 % (180–220, 108–132, 63–77 ms).
TC-CORE-03	Direction change via arrows and WASD	FR-003	Unit + System Functional Auto · AS	H	Snake heading right; default key bindings.	1. Queue Up (↑ and W), step; queue Down, step. 2. Repeat from heading up/down with Left/Right (← A, → D).	Heading changes to each perpendicular direction at the next tick for all 8 key/direction combinations.
TC-CORE-04	180° reversal rejected, incl. rapid double input	FR-003	Unit Negative / boundary Auto · AS	H	Heading right, length ≥ 3.	1. Queue Left; step. 2. Within one tick queue Up then Left; run two ticks.	Step 1: heading stays right, no game over. Step 2: Up applied on tick 1, Left on tick 2; head never enters the neck cell.
TC-CORE-05	Wall collision ends game (four walls)	FR-004	Unit Functional / boundary Auto · AS	H	Head placed on x=0 (heading left), x=29 (right), y=0 (up), y=19 (down).	1. Step once with head on the edge cell heading outward. 2. Repeat for one cell before the edge.	Game over (reason WALL) on the step leaving the grid; no game over while the head is still inside.
TC-CORE-06	Self collision ends game	FR-005	Unit Functional / negative Auto · AS	H	Length-5 snake in a U-shape fixture; length-4 snake in a 2×2 loop fixture.	1. Steer head into a body segment. 2. Steer head into the cell being vacated by the tail (not growing).	Step 1: game over (reason SELF). Step 2: no game over (tail-vacate rule, SAD OI-2).
TC-CORE-07	Food spawns on a free cell	FR-006	Unit Functional / boundary Auto · AS	H	Seeded RNG; auto-play harness; fixture with exactly 1 free cell.	1. Consume food 1 000 times; after each, inspect new food cell. 2. Run the 1-free-cell fixture; then a 0-free-cell fixture.	New food appears in the same tick, never on snake/obstacle/power-up; 1-free-cell case picks that cell; 0 free cells ends the session as “board full”.

ID	Title	Req.	Level / type / mode	Pri.	Preconditions & test data	Steps	Expected result
TC-CORE-08	Growth by exactly one segment	FR-007	Unit Functional Auto · AS	H	Length 3; food placed directly ahead.	1. Step onto food; observe length and tail. 2. Eat 10 consecutive foods; step on empty cells.	Length becomes 4 after first food (tail retained); 13 after ten; unchanged on non-food ticks.
TC-MODE-01	Mode selection	FR-008	System Functional Manual · BT	H	Mode/Difficulty screen reachable.	1. Open selection; list options. 2. Start one game in each mode.	Exactly three modes offered (Classic default). HUD shows chosen mode; timer only in Timed; obstacles only in Obstacle.
TC-MODE-02	Difficulty selection sets speed	FR-009, FR-002	Integration Functional Auto · BT	H	All 3×3 mode/difficulty combinations.	1. Start a session for each combination. 2. Read engine tick interval.	All 9 combinations start; tick interval equals 200 / 120 / 70 ms for Easy / Medium / Hard regardless of mode.
TC-MODE-03	Timed Challenge ends at zero	FR-010	Unit + System Functional / boundary Auto · BT	H	Timed mode, 60 s limit, fake Clock; snake kept alive by fixture.	1. Advance to 59.9 s. 2. Advance to 60.0 s.	Running at 59.9 s. Game over (TIME_UP) at 60.0 s (± 1 tick); HUD countdown never negative; final score retained.
TC-MODE-04	Obstacle placement and fatal collision	FR-011	Unit + System Functional / negative Auto · BT	H	Seeded RNG; Obstacle, Classic and Timed sessions.	1. Start Obstacle mode; list obstacle cells. 2. Steer head into an obstacle. 3. Start Classic and Timed; list obstacles.	8 obstacles on free cells ≥ 3 cells from head, not on food. Collision → game over (OBSTACLE). Classic/Timed: 0 obstacles.
TC-PWR-01	Power-up spawn interval	FR-012	Unit Functional / boundary Auto · BT	M	Fake Clock; seeded RNG; 10 simulated minutes.	1. Run session; record every power-up spawn instant and cell.	Every gap between spawns ≤ 15 s; spawn cell always unoccupied; at most one power-up on the grid.
TC-PWR-02	Speed boost lasts 5 s	FR-013	Unit + System Functional / timing Auto · BT	M	Medium difficulty (120 ms).	1. Collect Speed power-up; read tick interval each tick. 2. Advance clock 5 s.	Interval = 72 ms for 5.0 s (± 1 tick), then back to 120 ms; HUD shows remaining time.
TC-PWR-03	Score multiplier ×2 lasts 10 s	FR-014	Unit Functional / boundary Auto · BT	M	Classic mode (10 pts per food).	1. Collect Multiplier. 2. Eat food at t = 1 s, 9.9 s and 10.1 s.	Score deltas +20, +20, +10.
TC-PWR-04	Shrink and minimum length	FR-015	Unit Functional / boundary Auto · BT	M	Snakes of length 6, 4 and 3.	1. Collect Shrink at each length.	Lengths become 5, 3, 3; never below 3; power-up consumed each time.

ID	Title	Req.	Level / type / mode	Pri.	Preconditions & test data	Steps	Expected result
TC-PWR-05	Only one effect active; replacement	FR-013–015 + SRS 6.5	Unit Functional / negative Auto · BT	M	Fake Clock.	1. Collect Speed; after 2 s collect Multiplier. 2. Advance 10 s; then collect Speed again.	Speed reverts at the moment Multiplier is collected; only one effect ever active; no residual speed change after Multiplier expires.
TC-SCORE-01	Mode-specific score per food	FR-016	Unit Functional Auto · BH	H	Score 0 at session start.	1. Eat one food in each mode.	Classic +10, Timed +10, Obstacle +15; score never decreases.
TC-SCORE-02	Real-time score on HUD	FR-017	System Functional Manual · BH	H	Any mode; each theme.	1. Eat 5 foods while watching the HUD.	HUD value updates in the same frame as consumption and equals engine score; legible in all themes.
TC-LB-01	Persist Top-10 with all fields	FR-018	Integration Functional Auto · BH	H	Empty leaderboard; temp data directory.	1. Complete 5 games with different scores/modes. 2. Inspect leaderboard.json; restart app.	Valid JSON; sorted by score desc; every entry has player_name, score, game_mode, difficulty, date_achieved (YYYY-MM-DD); persists across restart.
TC-LB-02	View leaderboard	FR-019	System Functional Manual · BH	H	Fixtures: 10 entries; empty board.	1. Select Leaderboard from main menu with each fixture; press Esc.	Up to 10 rows (rank, name, score, mode, difficulty, date) in descending order; empty board shows “No scores yet”; Esc returns to menu.
TC-LB-03	Insert only if within Top-10	FR-020	Unit Boundary Auto · BH	H	Full board with 100, 200 … 1000; board with 3 entries.	1. Submit 1001, 101, 100, 99 on the full board. 2. Submit 1 on the 3-entry board.	1001 and 101 inserted (board stays 10, lowest dropped); 100 (tie with 10th) and 99 rejected with file unchanged; 1 accepted on the 3-entry board.
TC-LB-04	Missing/corrupt leaderboard fallback	FR-021, NFR-003	Integration Negative Auto · BH	M	Fixtures: missing, empty, truncated, wrong type, negative score, bad enum, 11 entries.	1. Launch and open Leaderboard for each fixture. 2. Finish a game with a valid score.	No crash; empty leaderboard shown; warning logged; valid result then stored (bad file replaced atomically).
TC-UI-01	Main menu	FR-022	System Functional Manual · DK	H	Fresh launch.	1. Observe main menu. 2. Navigate with keyboard and mouse; choose Exit.	Options exactly New Game, Leaderboard, Settings, Exit in this order; each opens its screen; Exit terminates the app.
TC-UI-02	Name entry limits	FR-023, SEC-006	System Boundary Auto+Manual · DK	H	Name Entry screen.	1. Enter 0, 1, 15, 16 and 40 characters; press Enter each time.	0 chars: inline error, cannot proceed. 15 accepted. 16th and later keystrokes ignored (field stays at 15). Name appears on leaderboard.
TC-UI-03	Pause and resume with Esc	FR-024	System Functional Manual+Auto · DK	H	Timed-mode game in progress with an active power-up.	1. Press Esc; wait 5 s. 2. Press Esc again.	Snake, timer and power-up timers frozen during pause; overlay shown; resume continues from identical state.

ID	Title	Req.	Level / type / mode	Pri.	Preconditions & test data	Steps	Expected result
TC-UI-04	Game Over screen	FR-025	System Functional Manual · DK	H	Game in progress.	1. Die by wall; read screen. 2. Choose Restart; die again; choose Main Menu.	Final score equals last HUD score; options Restart and Main Menu; Restart starts new session (same mode/difficulty, score 0); Main Menu returns to menu.
TC-SET-01	Remap movement keys	FR-026	System Functional / negative Manual+Auto · DK	M	Settings screen; default bindings.	1. Remap Up/Down/Left/Right to I/K/J/L; play. 2. Try assigning one key to two actions.	New keys steer the snake; old primary keys stop working (WASD alias still works); duplicate binding rejected with inline message.
TC-SET-02	Independent volume control	FR-027	System Functional / boundary Manual · DK	M	Music and SFX audible.	1. Set music 0 / SFX 100; play. 2. Set music 100 / SFX 0; play. 3. Try values below 0 and above 100.	Each slider affects only its channel, immediately; range clamped to 0–100.
TC-SET-03	Theme selection	FR-028	System Functional Manual · DK	L	Settings screen.	1. Select Classic, Neon, Retro; start a game each time.	Snake and grid colours/skin change with each theme; gameplay behaviour identical.
TC-SET-04	Settings persist across restarts	FR-029	Integration Functional Auto+Manual · DK	H	Non-default keys, volumes and theme set.	1. Exit via Exit; relaunch. 2. Inspect config.json.	All values restored; config.json matches SRS Appendix B layout (key_*, music_volume, sfx_volume, theme).
TC-PERF-01	Sustained ≥ 30 FPS	NFR-001	Performance Performance Auto · DK	H	Reference hardware profile (SAD/STP §6); Hard + Obstacle; power-ups active; 3 runs × 5 min.	1. Run the FPS probe for 5 minutes; log every 1-s window.	No 1-s window below 30 FPS in any run; min/avg/median reported.
TC-PERF-02	Input latency ≤ 100 ms	NFR-002	Performance Performance Auto · DK	H	Latency probe enabled; 100 key presses at random times.	1. Log key-event timestamp and timestamp when the engine registers the direction change.	Acknowledgement latency ≤ 100 ms for 100 % of presses; visible turn (next tick) recorded for information.
TC-REL-01	Missing/corrupt config fallback	NFR-003, SEC-003	Integration Negative Auto · DK	H	Fixtures: config missing, empty, truncated, out-of-range volume, bad theme.	1. Launch with each fixture; open Settings.	No crash; defaults applied (arrow keys, volumes 70/70, Classic); warning logged; config.json regenerated.
TC-USE-01	First game within 3 screens	NFR-004	Usability Usability Manual · DK	M	3 first-time volunteers outside the team; no instructions given.	1. Ask each to start a game; count screens before gameplay.	3 of 3 reach gameplay through Main Menu → Name Entry → Mode/Difficulty (≤ 3 screens) without help.

ID	Title	Req.	Level / type / mode	Pri.	Preconditions & test data	Steps	Expected result
TC-MNT-01	Module separation	NFR-005	Static / review Static analysis Auto+Review · BT	M	Repository at release candidate.	1. Run dependency-check script on includes. 2. Review module list against unit-test targets.	Logic/persistence contain 0 SFML includes; dependencies point downward only; ≥ 1 test suite each for engine, modes, persistence, UI logic.
TC-PORT-01	Windows 10/11 build and run	NFR-006	Build verification Portability Auto+Manual · BT	M	Clean clone; Windows 10 and Windows 11 machines.	1. cmake configure + build. 2. Launch; play one game; exit.	Build has 0 errors; app runs end-to-end on both OS versions. (Linux build informational only.)
TC-TEST-01	Headless testability	NFR-007	Unit / CI Testability Auto · BT	M	CI container without display.	1. Run engine/mode/score test binary; inspect its link line.	All tests run with no sf::Window created; test target does not link sfml-graphics/window; pass.
TC-SEC-01	Name sanitisation	SEC-001	Unit + System Negative / security Auto · BH	H	Inputs: `Ab,cd`, `a"b`, newline, tab, `{"x":1}`, `../x`, NUL, emoji, `ñ`; valid `A_b-1 x`.	1. Type/inject each input into the name field. 2. Complete a game and inspect leaderboard.json.	Every character outside [A-Za-z0-9 _-] is rejected with inline message; nothing unsafe written; valid name accepted.
TC-SEC-02	No PII stored	SEC-002	Inspection Design review Manual · BH	M	Several sessions played.	1. Inspect leaderboard.json, config.json and log; search for username, hostname and absolute paths.	Only SRS Appendix B fields present; no PII or absolute paths.
TC-SEC-03	Schema and size validation	SEC-003, SEC-007	Integration Negative / security Auto · BH	H	25 malformed fixtures per file: truncated, empty, binary, wrong types, out-of-range, unknown enum, extra fields, 10 000-level nesting, 64 KiB + 1 byte.	1. Load each fixture for both files under ASan/UBSan.	100 % handled: defaults used; no crash or hang (each load ≤ 1 s); no sanitiser report; oversize file rejected without full read.
TC-SEC-04	Safe write under fault injection	SEC-004	Integration Fault injection Auto · BH	H	Test hook aborts process (a) after temp opened, (b) after temp written, (c) after flush, (d) after rename; plus locked-target case.	1. Trigger a save with each fault; restart; validate file.	Restart shows old valid content for a–c and new valid content for d; never partial/empty; stale .tmp removed; locked target keeps old file and logs error.
TC-SEC-05	Untrusted data treated as data	SEC-005	Review + test Security review Auto+Review · BH	H	Hand-edited files containing `%s %n`, `$(calc)`, `<script>`, `${PATH}`.	1. Load and display the entries. 2. Run CI grep for system/popen/exec*/dlopen and printf-family with file-derived format.	Strings shown literally or entry rejected; no command executed; 0 banned-API hits.
TC-SEC-06	Buffer limit on name input	SEC-006	Unit + System Boundary / security Auto · BH	H	ASan/UBSan build; automated event injection.	1. Inject 10 000 text-entered events (and one 10 000-char string to the validator).	Stored name ≤ 15 chars; no crash; no sanitiser report; memory stays stable.